claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-acf43291b6109fb73.jsonl`
- SHA-256: `ec02d1ca13b87b74c6ea359990a73a070a41ed6f174c4c1ff2a19757cca5e782`
- Source modified: `2026-06-19T15:49:31+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_ac0cbb29-6bf`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`

Transcript

1. user (2026-06-19T15:46:34.351Z)

```
You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements
that CONTRADICT the current code or data.
HARD RULES:
- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open
the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured,
say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate
owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy
choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST
be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.
Working dir is the repo root; use Read/Grep/Glob freely.

YOUR SLICE ? eval:
Eval/metric/windowing doc staleness vs code: docs/EVALUATION.md, docs/R2_EVAL_METRIC_DESIGN.md,
docs/EXPERIMENT_HARNESS.md, docs/RALLY_QUALITY_RESEARCH.md, docs/R1_MILESTONE_LAUNCH_REPORT.md.
Verify against: backend/eval/windowing.py (the SERVED preset = the milestone now),
backend/eval/tolerance_metrics.py (R2), backend/eval/rally_seg_eval.py (flags incl the new
--min-duration/--stratify/--long-tau from #220), backend/config/models.py (#204 milestone
defaults). Report doc claims contradicted by the code (e.g. served operating point described as
pre-flip, missing R2/long-rally lens, wrong default values).

Return EVERY code-contradicting staleness item you can verify in your slice. Exhaustive within
the slice; facts only.
```

2. assistant (2026-06-19T15:46:37.045Z)

```
I'll audit the eval docs against the code. Let me start by reading the key code files (the
source of truth) and the docs in parallel.
```

3. user (2026-06-19T15:46:38.006Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
```

```
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
10      exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11      (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13      **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14      SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15      (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16      build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17      *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18      operating point a given deployment truly serves.
19
20      The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21      windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22      ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24      Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25      caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26      """
27      from __future__ import annotations
28
29      from dataclasses import dataclass
30      from typing import Any, Dict, List, Mapping, Tuple
31
32      from backend.config.models import IndexingConfig
33
34      Interval = Tuple[float, float]
35
36      __all__ = [
37          "WindowingPreset",
38          "SERVED",
39          "PROPOSAL",
40          "PRESETS",
41          "served_windowing_from_config",
42          "build_windows",
43          "windows_to_intervals",
44      ]
45
46
47      @dataclass(frozen=True)
48      class WindowingPreset:
49          """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51          ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52          "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53          ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54          knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55          """
```

```
56
57          name: str
58          velocity_thresh: float
59          merge_gap: float
60          min_window_duration: float
61          #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62          #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63          max_window_duration: float = 0.0
64          #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65          #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66          hard_cap: bool = False
67          #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68          #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69          inactivity_end: float = 0.0
70          #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71          inactivity_rally_thresh: float = 0.08
72          inactivity_min_density: float = 0.34
73          #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74          #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75          #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76          #: factor keeps it surgical (a normal long rally is left alone).
77          blob_fallback: bool = False
78          blob_factor: float = 4.0
79
80          def as_tuple(self) -> Tuple[float, float, float]:
81              """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82              ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83              bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84              return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86          def to_dict(self) -> Dict[str, Any]:
87              return {
88                  "name": self.name,
89                  "velocity_thresh": self.velocity_thresh,
90                  "merge_gap": self.merge_gap,
91                  "min_window_duration": self.min_window_duration,
92                  "max_window_duration": self.max_window_duration,
93                  "hard_cap": self.hard_cap,
94                  "inactivity_end": self.inactivity_end,
95                  "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                  "inactivity_min_density": self.inactivity_min_density,
97                  "blob_fallback": self.blob_fallback,
98                  "blob_factor": self.blob_factor,
99              }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106           - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
```

```
wasb/fusion/tracknet)
107           - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108           - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112         so the two can never silently diverge again.
113         """
114         idx = IndexingConfig()  # construct with defaults ? the served operating point
115         return WindowingPreset(
116             name="served",
117             velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
118             merge_gap=float(idx.dead_time_merge_gap),
119             min_window_duration=float(idx.min_action_window_duration),
120             max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
(default 0=OFF)
121             hard_cap=bool(idx.window_hard_cap),                   # W1 hard-cap fallback
(default OFF)
122             inactivity_end=float(idx.window_inactivity_end),      # R4 rally-end trim
(default 0=OFF)
123             inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124             inactivity_min_density=float(idx.inactivity_min_density),
125             blob_fallback=bool(idx.window_blob_fallback),         # R5 blob-fallback
(default OFF)
126             blob_factor=float(idx.window_blob_factor),
127         )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
```

```
        idx = _as_mapping(config, "indexing")
        family_cfg = _as_mapping(idx, family)
        return WindowingPreset(
            name=f"served:{family}",
            velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
            merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
            min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
            max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
            hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
            inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
            inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
            inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
            blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
            blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
        )


    def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
        """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
        (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
        if obj is None:
            return {}
        if isinstance(obj, Mapping):
            sub = obj.get(key, {})
        elif hasattr(obj, "get"):
            sub = obj.get(key, {})
        else:
            sub = getattr(obj, key, {})
        return sub if isinstance(sub, Mapping) else {}


    def build_windows(points: List[Any], fps: float, frame_width: float,
                      preset: WindowingPreset) -> List[Dict[str, Any]]:
        """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).

        A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
        instead of three loose floats, so eval candidate-building and the served path
reference the
        SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

        vt, mg, mwd = preset.as_tuple()
        return TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            max_window_duration=preset.max_window_duration,
            window_hard_cap=preset.hard_cap,
            window_inactivity_end=preset.inactivity_end,
            inactivity_rally_thresh=preset.inactivity_rally_thresh,
            inactivity_min_density=preset.inactivity_min_density,
            window_blob_fallback=preset.blob_fallback,
            window_blob_factor=preset.blob_factor)


    def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
```

```
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211
```

4. user (2026-06-19T15:46:38.514Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
16      """
17      from __future__ import annotations
18
19      import statistics
20      from typing import Any, Dict, List, Optional, Tuple
21
22      Interval = Tuple[float, float]
23      #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24      Match = Tuple[int, int, float, float]
25
26      #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
27      #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
28      TAU_START: float = 2.0
29      TAU_END: float = 1.5
30      SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
31
32
33      def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
34          return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
35
36
37      def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
38                             tau_start: float, tau_end: float) -> List[Match]:
39          """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
40          assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
41          graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
42          pairs: List[Tuple[float, float, int, int]] = []
43          for i, p in enumerate(preds):
44              for j, g in enumerate(gts):
45                  if _eligible(p, g, tau_start, tau_end):
```

```
                    # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
                    pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
        pairs.sort()
        used_p: set = set()
        used_g: set = set()
        matches: List[Match] = []
        for _, _, i, j in pairs:
            if i in used_p or j in used_g:
                continue
            used_p.add(i)
            used_g.add(j)
            matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
                              tau_start: float, tau_end: float) -> Optional[List[Match]]:
        """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
        numpy/scipy are unavailable so the caller falls back to greedy."""
        try:
            import numpy as np
            from scipy.optimize import linear_sum_assignment
        except Exception:
            return None
        n, m = len(preds), len(gts)
        size = max(n, m)
        big = 1.0e6
        cost = np.full((size, size), big, dtype=float)
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
        rows, cols = linear_sum_assignment(cost)
        matches: List[Match] = []
        for i, j in zip(rows.tolist(), cols.tolist()):
            if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
                matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def match_1to1(preds: List[Interval], gts: List[Interval],
                   tau_start: float = TAU_START, tau_end: float = TAU_END
                   ) -> Tuple[List[Match], float, float, float]:
        """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
        (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
        present; deterministic greedy otherwise."""
        n, m = len(preds), len(gts)
        if n == 0 or m == 0:
            return [], 0.0, 0.0, 0.0
        matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
        if matches is None:
            matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
        k = len(matches)
        precision = k / n
        recall = k / m
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
        return matches, precision, recall, f1
```

```
105
106
107     def _percentile(xs: List[float], p: float) -> float:
108         """Linear-interpolated percentile (pure stdlib)."""
109         if not xs:
110             return 0.0
111         s = sorted(xs)
112         k = (len(s) - 1) * p / 100.0
113         lo = int(k)
114         hi = min(lo + 1, len(s) - 1)
115         return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118     def _overlap(a: Interval, b: Interval) -> float:
119         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122     def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124         end open" diagnostic that aims R4.
125
126         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128         ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131         def dist(vals: List[float]) -> Dict[str, float]:
132             av = [abs(v) for v in vals]
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []
142         for g in gts:
143             best = max(preds, key=lambda p: _overlap(p, g), default=None)
144             if best is not None and _overlap(best, g) > 0.0:
145                 ds.append(best[0] - g[0])
146                 de.append(best[1] - g[1])
147         return {"dstart": dist(ds), "dend": dist(de)}
148
149
150     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151         """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
152         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
153         overlap-graph in ``segmentation_metrics.merge_split_report``."""
154         from backend.eval.segmentation_metrics import merge_split_report
155         r = merge_split_report(preds, gts)
156         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
157
158
159     # ---------------------------------------------------------------------- #
160     # R1 ? net over-segmentation promotion guard
```

```
    # ---------------------------------------------------------------------- #
    #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
    #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
    #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
    #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
    #: the asymmetry is the principled default, not a number tuned to force a verdict.)
    WEIGHT_MERGE: float = 2.0
    WEIGHT_SPLIT: float = 1.0


    def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
                       weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
                       use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
        """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).

        ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
        emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
        ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
        position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.

        **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
        ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
        windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
        (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
        ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
        ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
        that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
        So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
        over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
        ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)

        **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
        require BOTH:
          1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
          2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
             of a video's rallies is the one over-merge regression that is never acceptable,
the honest
             "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
        With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
        affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
```

```
200
201         **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202         strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203         """
204         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205             if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206                 cmap = {c["name"]: c for c in cand}
207                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211         def _cost(row: Dict[str, Any]) -> float:
212             if use_rates:
213                 return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216         pairs = _aligned()
217         n = len(pairs)
218         base_costs = [_cost(b) for b, _ in pairs]
219         cand_costs = [_cost(c) for _, c in pairs]
220         base_net = (sum(base_costs) / n) if n else 0.0
221         cand_net = (sum(cand_costs) / n) if n else 0.0
222         net_delta = cand_net - base_net
223
224         # Per-video regressions, reported for transparency.
225         new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
226         new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
227         merge_rate_regress: List[Dict[str, Any]] = []    # RATE increases (the real over-
merge regression)
228         for b, c in pairs:
229             if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230                 new_merges.append({"name": b.get("name", ""),
231                                    "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232             if float(c["split_count"]) > float(b["split_count"]) + eps:
233                 new_splits.append({"name": b.get("name", ""),
234                                    "base": float(b["split_count"]), "cand":
float(c["split_count"])})
235             if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236                 merge_rate_regress.append({"name": b.get("name", ""),
237                                            "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})
238
239         strict_passes = (not new_merges) and (not new_splits)
240         net_cost_ok = net_delta <= eps
241         no_merge_rate_regress = not merge_rate_regress
242         passes = bool(net_cost_ok and no_merge_rate_regress)
243         return {
244             "passes": passes,
245             "strict_passes": strict_passes,
246             "net_cost_ok": net_cost_ok,
247             "no_merge_rate_regress": no_merge_rate_regress,
248             "base_net": base_net,
249             "cand_net": cand_net,
250             "net_delta": net_delta,
251             "n": n,
252             "use_rates": use_rates,
253             "weight_merge": weight_merge,
```

```
254             "weight_split": weight_split,
255             "new_merges": new_merges,
256             "new_splits": new_splits,
257             "merge_rate_regress": merge_rate_regress,
258         }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                          tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                         taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278
```

5. user (2026-06-19T15:46:39.512Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
```

```
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73          keep_frames: bool = False
74          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75          min_free_gb: float = 0.0
76          # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77          # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81          stream_video: bool = False
82
83
84      class FusionConfig(BaseModel):
85          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87          Every default reproduces control behaviour: the fusion segmenter persists the
88          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89          true ? the person-cue features, but it does NOT gate the served handoff unless a
```

```
90          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92          supplied ? off-court persons (spectators / adjacent courts) are excluded.
93          """
94          # Which trajectory substrate seeds the windows (shuttle stays primary).
95          substrate: Literal["wasb", "tracknet"] = "wasb"
96          # Windowing velocity threshold for the fusion family (the engine reads
97          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99          velocity_threshold: float = 0.02
100         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101         # enabled ? so the default path never imports torch / touches the GPU.
102         cue_flags: dict[str, bool] = Field(default_factory=dict)
103         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105         min_crossings: int = Field(default=0, ge=0)
106         # Served Gate B: when the person cue is on, drop windows with median in-court
players
107         # < this. 0 = OFF.
108         min_players: int = Field(default=0, ge=0)
109         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110         court_polygon: Optional[list[list[float]]] = None
111         # Net line for the person two-sided-motion split (person features only ? the shuttle
112         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113         net_axis: Literal["x", "y"] = "y"
114         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
133         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
134         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139         max_window_duration: float = 6.0
140         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
141         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
```

```
inactivity gap
142         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144         # Only cuts (recall can't drop). OFF by default until measured/promoted.
145         window_hard_cap: bool = False
146         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
147         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
150         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
152         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153         window_inactivity_end: float = 1.5
154         inactivity_rally_thresh: float = 0.20
155         inactivity_min_density: float = 0.30
156         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
158         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
161         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164         window_blob_fallback: bool = False
165         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169         window_blob_factor: float = 4.0
170         log_yolo_candidates: bool = True
171         store_yolo_candidates: bool = True
172         ai_handoff_padding: float = 2.0
173         ai_max_workers: int = 5
174         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
175         # chunks of a high-bitrate (4K) source blow past the provider upload cap
176         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
177         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
178         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
179         # Matches gemini_refine's --clip-scale-width default.
180         ai_chunk_scale_width: int = 1280
181         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
182         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
183         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
```

```
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
```

```
241         end_padding: float = 0.5
242         use_cache: bool = False
243         min_shot_count: int = 1
244         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
245         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
246         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
247         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
248         # local detector over-fires and its false positives are mostly SHORT (motion blips,
249         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
250         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
251         min_rally_duration: float = Field(default=0.0, ge=0.0)
252         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
253
254
255     class LoggingConfig(BaseModel):
256         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
257         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
258         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
259         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
260         # them to WARNING; set true to restore full chatter for request-flow debugging.
261         verbose_http: bool = False
262
263
264     class SecurityConfig(BaseModel):
265         # When set, /api/process video_path must resolve under this directory (hosted/tenant
266         # mode). Default None preserves the single-user local 'any local file' workflow.
267         allowed_video_root: Optional[str] = None
268
269         # --- Chunked upload (B2, PR-2) ---
270         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
271         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
272         # contain upload_dir or /api/process will reject the uploaded paths.
273         upload_dir: str = "output/uploads"
274         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
275         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
276         max_upload_mb: int = 4096
277         max_part_mb: int = 95
278         allowed_upload_extensions: List[str] = ["mp4", "mov"]
279
280
281     class AppConfig(BaseModel):
282         """Root configuration object.
283
284         All runtime configuration should be accessed via an instance of this class
285         (or the loader) rather than raw dict lookups.
286         """
287
288         sport: Sport = "badminton"
289         ai_provider: str = "gemini"
290         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
291         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
292         ai_model: str = "gemini-flash-latest"
293
294         validation: ValidationConfig = Field(default_factory=ValidationConfig)
295         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
296         output: OutputConfig = Field(default_factory=OutputConfig)
297         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
298         security: SecurityConfig = Field(default_factory=SecurityConfig)
```

```
299         logging: LoggingConfig = Field(default_factory=LoggingConfig)
300
301         # Allow extra keys during transition so existing config.json files do not break.
302         model_config = {
303             "extra": "allow",
304             "validate_assignment": True,
305         }
306
307         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
308         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
309         # attribute access / the typed config. A nested *section* is returned as a plain
dict
310         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
311         # section model instead breaks it (a model has no .get), which bit every validator
on
312         # real runs. Scalar leaves are returned as-is.
313         def get(self, key: str, default: Any = None) -> Any:
314             _MISSING = object()
315             val: Any = _MISSING
316             if key in type(self).model_fields or hasattr(self, key):
317                 val = getattr(self, key, _MISSING)
318             if val is _MISSING:
319                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
320                 # discovered dynamically from the model fields, so a future top-level
section
321                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
322                 for fname in type(self).model_fields:
323                     section = getattr(self, fname, None)
324                     if isinstance(section, BaseModel) and hasattr(section, key):
325                         val = getattr(section, key)
326                         break
327             if val is _MISSING:
328                 return default
329             if isinstance(val, BaseModel):
330                 return val.model_dump(mode="json")
331             return val
332
333         def __getitem__(self, key: str) -> Any:
334             val = self.get(key)
335             if val is None and key not in type(self).model_fields:
336                 # Re-raise like a real dict for unknown top-level
337                 raise KeyError(key)
338             return val
339
340
341     # Convenience type alias for callers that want to annotate
342     Config = AppConfig
343
```

6. user (2026-06-19T15:46:39.856Z)

```
1       """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2       over-segmentation-sensitive metric set, honestly.
3
4       This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 /
§7)
5       measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6       IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7       segment-count ratio, and the explicit ``merge_split`` breakdown
```

(``segmentation_metrics``).

      For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds candidate
      windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED
      operating point and any future windowing the same way) ? scores the windows-as-rallies against
      the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI, and can
      diff two windowings with a paired sign-test + CI (the per-tier ``delta``).

      GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each video's
      trajectory path + fps + frame_width; the GT rally intervals come from
      ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label files.

      Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
      """
      from __future__ import annotations

      import argparse
      import json
      import os
      from typing import Any, Dict, List, Optional, Tuple

      from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
      from backend.eval.metrics import Interval
      from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

      DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
      DEFAULT_FEATURES_DIR = "output"
      DEFAULT_IOU = 0.5

      #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its duration
      #: (end ? start) exceeds this. The local detector over-fires, and its false positives are mostly
      #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to long rallies
      #: strips most of those FPs and reveals the real product quality on the long-rally happy path. Used
      #: as the default split point for the stratified report (``--long-tau`` overrides it). DURATION, not
      #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path reports it as
      #: "Unknown", whereas duration is derived from the same start/end labels we already trust.
      LONG_RALLY_TAU = 5.0

      #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden manifest
      #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the AUDITABLE source
      #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the clips tagged
      #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the dominant
      #: source of short false positives ? so we report it separately. A manifest entry MAY carry an
      #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
      MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"})

```
53
54      def court_type_for(video: Dict[str, Any]) -> str:
55          """``'multicourt'`` or ``'single'`` for a golden video. Honors an explicit manifest
56          ``court_type`` field when present, else falls back to membership in
``MULTICOURT_CLIPS``
57          (keyed by ``name``), else ``'single'``."""
58          ct = video.get("court_type")
59          if isinstance(ct, str) and ct:
60              return ct
61          return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64      def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65          """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
66
67          ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68          positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69          to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
70          if min_duration <= 0.0:
71              return list(ivs)
72          return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75      # ---------------------------------------------------------------------------- #
76      # Pure scoring core (no I/O ? unit-testable)
77      # ---------------------------------------------------------------------------- #
78      def score_intervals(preds: List[Interval], gts: List[Interval],
79                          iou: float = DEFAULT_IOU,
80                          tau_start: Optional[float] = None,
81                          tau_end: Optional[float] = None,
82                          min_duration: float = 0.0) -> Dict[str, Any]:
83          """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85          Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
86          over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87          merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89          Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
90          tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
92          start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94          ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95          and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96          row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
97          """
98          from backend.eval import tolerance_metrics as tm
99          preds = filter_by_min_duration(preds, min_duration)
100         gts = filter_by_min_duration(gts, min_duration)
101         ts = tm.TAU_START if tau_start is None else tau_start
102         te = tm.TAU_END if tau_end is None else tau_end
103         sc = metrics.score(preds, gts, iou_threshold=iou)
104         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
```

```
105         ms = segmentation_metrics.merge_split_report(preds, gts)
106         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107         be = tm.boundary_error_report(preds, gts)  # unconditional best-overlap (the
R4-aiming diagnostic)
108         osr = tm.over_seg_report(preds, gts)
109         return {
110             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
111             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
112             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
113             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
114             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
115             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
116             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
117             # --- R2 (tolerance-match) ? the task-faithful headline block ---
118             "tau_start": ts, "tau_end": te,
119             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
120             "tol_matched": float(len(tol_matches)),
121             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
122             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
123         }
124
125
126     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
127                  "precision", "recall",
128                  "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
129
130
131     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
132         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
133
134         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
135         leave-one-video-out discipline (windows within a video are correlated)."""
136         out: Dict[str, Any] = {"n": len(rows)}
137         for k in _AGG_KEYS:
138             vals = [r[k] for r in rows]
139             out[k] = eval_stats.mean_with_ci(vals) if vals else None
140         return out
141
142
143     # ------------------------------------------------------------------------ #
144     # Golden corpus I/O
145     # ------------------------------------------------------------------------ #
146     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
148         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
149         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
150         with open(manifest_path) as fh:
151             manifest = json.load(fh)
152         out: List[Dict[str, Any]] = []
153         for v in manifest:
154             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
155             gts: List[Interval] = []
156             if os.path.exists(feat):
157                 with open(feat) as fh:
158                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
159             out.append({"name": v["name"], "trajectory": v["trajectory"],
```

```
                        "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
        return out


    def windows_for(video: Dict[str, Any], preset: WindowingPreset,
                    min_crossings: int = 0) -> List[Interval]:
        """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.

        ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
        ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
        times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
        from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
        windowing (W1) leaves behind.

        NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
        drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
        strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
        """
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
        points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
        wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
        ivs = windowing.windows_to_intervals(wins)
        if min_crossings > 0 and ivs:
            from backend.pipeline.detectors.rally_gate import gate_windows
            gated = [(g.start, g.end)
                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
                     if g.kept]
            # Gating emptied a non-empty video ? keep the ungated windows.
            ivs = gated if gated else ivs
        return ivs


    #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
    PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]


    def predict_all(golden: List[Dict[str, Any]], preset: WindowingPreset,
                    min_crossings: int = 0) -> List["PredictedVideo"]:
        """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
        returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
        predictions under several lenses (e.g. the all-vs-long stratified report) without
re-windowing."""
        out: List[PredictedVideo] = []
        for v in golden:
            if not v["gts"] or not os.path.exists(v["trajectory"]):
                continue
            out.append((v["name"], court_type_for(v), windows_for(v, preset, min_crossings),
v["gts"]))
        return out


    def _score_predicted(predicted: List["PredictedVideo"], iou: float,
                         tau_start: Optional[float], tau_end: Optional[float],
```

```
                         min_duration: float) -> List[Dict[str, Any]]:
        """Score pre-windowed videos into per-video rows (carrying ``name`` +
``court_type``)."""
        rows: List[Dict[str, Any]] = []
        for name, court, preds, gts in predicted:
            row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
            row["name"] = name
            row["court_type"] = court
            rows.append(row)
        return rows


    def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
                 min_duration: float = 0.0) -> Dict[str, Any]:
        """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
        rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block, plus
        its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs
filtered to
        rallies longer than that many seconds) consistently across both metric paths."""
        rows = _score_predicted(predict_all(golden, preset, min_crossings),
                                iou, tau_start, tau_end, min_duration)
        return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
                "min_duration": min_duration, "rows": rows, "aggregate": aggregate(rows)}


    def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
                iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
                cand_min_crossings: int = 0,
                tau_start: Optional[float] = None, tau_end: Optional[float] = None,
                guard_weight_merge: Optional[float] = None,
                guard_weight_split: Optional[float] = None,
                guard_use_rates: bool = True, min_duration: float = 0.0) -> Dict[str, Any]:
        """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
        sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
        over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
        the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
        alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
        long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
        knob, so it is identical on both sides of the A/B)."""
        from backend.eval import tolerance_metrics as tm
        b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end,
min_duration)
        c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end,
min_duration)
        by_name_b = {r["name"]: r for r in b["rows"]}
        paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
        delta: Dict[str, Any] = {}
        for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
            delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
                                                  [rc[k] for _, rc in paired])
        gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
        if guard_weight_merge is not None:
            gkw["weight_merge"] = guard_weight_merge
        if guard_weight_split is not None:
```

```
263             gkw["weight_split"] = guard_weight_split
264         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
265         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
266
267
268     def stratified_report(golden: List[Dict[str, Any]], preset: WindowingPreset,
269                           iou: float = DEFAULT_IOU, min_crossings: int = 0,
270                           tau_start: Optional[float] = None, tau_end: Optional[float] =
None,
271                           long_tau: float = LONG_RALLY_TAU) -> Dict[str, Any]:
272         """The long-rally lens payoff: ``{all vs long(>long_tau)} × {single vs multicourt}``
aggregates.
273
274         The thesis: local OVER-FIRES, and its false positives are mostly SHORT (motion
blips,
275         background-court fragments on multicourt footage). Filtering to long rallies should
strip those
276         FPs ? precision (``tol_precision``) should jump, most on multicourt. This table is
how we read
277         "do we beat Gemini on the single-match long-rally happy path?".
278
279         Windowing is run ONCE per video (``predict_all``); each cell re-scores the same
predictions
280         under its duration lens. A cell aggregates only videos with >=1 golden rally in that
stratum
281         (``num_gt`` > 0): a clip with no long rally is not evidence about long-rally quality
and its
282         0/0 recall would distort the mean. ``n`` per cell reports the videos that survived
that gate."""
283         predicted = predict_all(golden, preset, min_crossings)
284         long_label = f"long>{long_tau:g}s"
285         strata = (("all", 0.0), (long_label, long_tau))
286         courts = ("single", "multicourt", "all")
287         cells: Dict[Tuple[str, str], Dict[str, Any]] = {}
288         for dur_label, min_dur in strata:
289             rows = _score_predicted(predicted, iou, tau_start, tau_end, min_dur)
290             for court in courts:
291                 sub = [r for r in rows
292                        if (court == "all" or r["court_type"] == court) and r["num_gt"] > 0]
293                 cells[(dur_label, court)] = aggregate(sub)
294         return {"long_tau": long_tau, "labels": [s[0] for s in strata], "courts":
list(courts),
295                 "cells": cells, "preset": preset.to_dict(), "iou": iou, "min_crossings":
min_crossings}
296
297
298     # ------------------------------------------------------------------------ #
299     # Reporting
300     # ------------------------------------------------------------------------ #
301     def _ci(m: Optional[Dict[str, Any]]) -> str:
302         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
303
304
305     def _format_guard(g: Dict[str, Any]) -> str:
306         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
307         basis = "rate" if g["use_rates"] else "count"
308         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
309                  f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
310                  f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
311                  f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
312                  f"no_merge_rate_regress={g['no_merge_rate_regress']})",
313                  f"  strict per-video rule (R2 §2.3.3, for contrast): "
```

```
                f"{'pass' if g['strict_passes'] else 'BLOCK'}  "
                f"(count increases ? merges:{len(g['new_merges'])}
splits:{len(g['new_splits'])})"]
        if g["merge_rate_regress"]:
            names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
                              for r in g["merge_rate_regress"])
            lines.append(f"  ? merge_rate REGRESSED on: {names}")
        return "\n".join(lines)


    def format_report(result: Dict[str, Any]) -> str:
        p = result["preset"]
        bounded = []
        if p.get("max_window_duration"):
            bounded.append(f"cap={p['max_window_duration']}" + ("(hard)" if
p.get("hard_cap") else ""))
        if p.get("inactivity_end"):
bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
                           f"/md={p['inactivity_min_density']}")
        if p.get("blob_fallback"):
            bounded.append("blob_fallback")
        extra = (", " + ", ".join(bounded)) if bounded else ""
        md = result.get("min_duration", 0.0) or 0.0
        md_tag = f", LONG-RALLY LENS min_dur>{md:g}s" if md > 0 else ""
        lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}{md_tag}",
""]
        lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
        lines.append("|---|---|---|---|---|---|---|---|")
        for r in result["rows"]:
            lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
                         f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
                         f"{r['num_pred']}/{r['num_gt']} |")
        a = result["aggregate"]
        lines += ["", f"**Aggregate (n={a['n']}, mean [95% CI]):** "
                  f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
{_ci(a['f1@0.5'])} · "
                  f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
        # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
        rows = result["rows"]
        if rows and "tol_f1" in rows[0]:
            ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
            lines += ["", f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
                      "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
                      "|---|---|---|---|---|---|---|---|"]
            for r in rows:
                lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
                             f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
                             f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
            lines += ["", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
                      f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
                      f"|?start| {_ci(a['dstart_abs_median'])} · |?end|
{_ci(a['dend_abs_median'])} "
                      f"(start?solved / end=the gap)"]
        return "\n".join(lines)
```

```
365
366
367     def format_stratified(strat: Dict[str, Any]) -> str:
368         """Render the long-rally stratified table: {all vs long} × {single, multicourt, all-
courts}.
369
370         ``tolP`` is the column that carries the story ? if filtering to long rallies strips
local's
371         short false positives, precision rises (most on multicourt). ``n`` is the videos
with >=1 golden
372         rally in that stratum."""
373         long_tau = strat["long_tau"]
374         cells = strat["cells"]
375         lines = [f"\n## Long-rally lens ? stratified (long = duration > {long_tau:g}s; "
376                  "n = videos with >=1 golden rally in the stratum)", "",
377                  "| stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |",
378                  "|---|---|---|---|---|---|---|---|"]
379         for dur_label in strat["labels"]:
380             for court in strat["courts"]:
381                 a = cells[(dur_label, court)]
382                 lines.append(f"| {dur_label} | {court} | {a['n']} | {_ci(a['tol_f1'])} | "
383                              f"{_ci(a['tol_precision'])} | {_ci(a['tol_recall'])} | "
384                              f"{_ci(a['f1@0.5'])} | {_ci(a['segment_count_ratio'])} |")
385         return "\n".join(lines)
386
387
388     def _resolve_preset(name: str) -> WindowingPreset:
389         if name in PRESETS:
390             return PRESETS[name]
391         raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
392
393
394     def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
395         """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
``args`` by
396         applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
397         inactivity. ``cand`` falls back to the base override when its ``--vs-*`` variant is
unset
398         (so a baseline-vs-candidate promotion check runs in one shot)."""
399         from dataclasses import replace
400         if side == "base":
401             preset = _resolve_preset(args.preset)
402             cap = args.max_window_duration
403             hard_cap = args.hard_cap
404             blob_fallback = args.blob_fallback
405             inactivity_end = args.inactivity_end
406         else:
407             preset = _resolve_preset(args.vs)
408             cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
409                    else args.max_window_duration)
410             hard_cap = args.vs_hard_cap or args.hard_cap
411             blob_fallback = args.vs_blob_fallback or args.blob_fallback
412             inactivity_end = (args.vs_inactivity_end if args.vs_inactivity_end is not None
413                               else args.inactivity_end)
414         if cap is not None:
415             preset = replace(preset, max_window_duration=cap)
416         if hard_cap:
417             preset = replace(preset, hard_cap=True)
418         if blob_fallback:
419             preset = replace(preset, blob_fallback=True)
420         if args.blob_factor is not None:
421             preset = replace(preset, blob_factor=args.blob_factor)
422         if inactivity_end is not None:
423             preset = replace(preset, inactivity_end=inactivity_end)
424         if args.inactivity_rally_thresh is not None:
```

```
425             preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
426         if args.inactivity_min_density is not None:
427             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
428         return preset
429
430
431     def main() -> None:
432         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
433         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
434         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
435         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
436         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
437         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
438         ap.add_argument("--min-crossings", type=int, default=0,
439                         help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
440         ap.add_argument("--vs-min-crossings", type=int, default=0,
441                         help="net-crossings gate on --vs (default: same as --min-
crossings)")
442         ap.add_argument("--max-window-duration", type=float, default=None,
443                         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
444         ap.add_argument("--vs-max-window-duration", type=float, default=None,
445                         help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
446                              "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
447         ap.add_argument("--hard-cap", action="store_true",
448                         help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
449                              "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
450         ap.add_argument("--vs-hard-cap", action="store_true",
451                         help="hard-cap fallback on --vs (default: same as --hard-cap)")
452         ap.add_argument("--blob-fallback", action="store_true",
453                         help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
454                              "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
455         ap.add_argument("--vs-blob-fallback", action="store_true",
456                         help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
457         ap.add_argument("--blob-factor", type=float, default=None,
458                         help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
459         ap.add_argument("--inactivity-end", type=float, default=None,
460                         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
461         ap.add_argument("--vs-inactivity-end", type=float, default=None,
462                         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
463         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
464                         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
465         ap.add_argument("--inactivity-min-density", type=float, default=None,
466                         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
467         ap.add_argument("--tau-start", type=float, default=None,
468                         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
469         ap.add_argument("--tau-end", type=float, default=None,
470                         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
471         ap.add_argument("--guard-weight-merge", type=float, default=None,
```

```
                        help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
        ap.add_argument("--guard-weight-split", type=float, default=None,
                        help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
        ap.add_argument("--guard-counts", action="store_true",
                        help="R1 over-seg guard scores raw counts instead of normalized
rates "
                             "(rates are the default ? comparable across a structural
mega?bounded change)")
        ap.add_argument("--min-duration", type=float, default=0.0,
                        help="LONG-RALLY LENS (0=OFF): score ONLY rallies longer than this
many seconds, "
                             "filtering BOTH predictions and golden GTs before matching.
Strips local's "
                             "short false-positive blips to reveal long-rally quality.
Applies across the "
                             "tIoU + R2 paths and to --vs (same filter on both sides).")
        ap.add_argument("--stratify", action="store_true",
                        help="also print the long-rally stratified report: {all vs
long(>--long-tau)} × "
                             "{single vs multicourt} aggregates (court type from
GOLDEN_VIDEOS.md / the "
                             "manifest court_type field). The actual long-rally payoff
table.")
        ap.add_argument("--long-tau", type=float, default=LONG_RALLY_TAU,
                        help=f"the 'long' rally duration threshold (s) used by --stratify "
                             f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is
>5s).")
        ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
        args = ap.parse_args()

        golden = load_golden(args.manifest, args.features_dir)
        base = _build_preset(args, "base")
        if args.vs:
            vs_mc = args.vs_min_crossings or args.min_crossings
            cand = _build_preset(args, "cand")
            result = compare(golden, base, cand, args.iou,
                             base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
                             tau_start=args.tau_start, tau_end=args.tau_end,
                             guard_weight_merge=args.guard_weight_merge,
                             guard_weight_split=args.guard_weight_split,
                             guard_use_rates=not args.guard_counts,
                             min_duration=args.min_duration)
            print(format_report(result["base"]))
            print("\n" + format_report(result["cand"]))
            d = result["delta"]
            print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
                  f"n_paired={result['n_paired']}")
            for k, dd in d.items():
                st = dd["sign_test"]
                print(f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
                      f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
            print(_format_guard(result["over_seg_guard"]))
        else:
            result = evaluate(golden, base, args.iou, args.min_crossings,
                              tau_start=args.tau_start, tau_end=args.tau_end,
                              min_duration=args.min_duration)
            print(format_report(result))
        if args.stratify:
            # The long-rally payoff table always reports the BASE preset (the served/current
```

```
config),
522             # split by duration × court type. Independent of --min-duration (it does its own
split).
523             print(format_stratified(stratified_report(
524                 golden, base, args.iou, args.min_crossings,
525                 tau_start=args.tau_start, tau_end=args.tau_end, long_tau=args.long_tau)))
526         if args.json_out:
527             with open(args.json_out, "w", encoding="utf-8") as fh:
528                 json.dump(result, fh, indent=2)
529             print(f"\n[written {args.json_out}]")
530
531
532     if __name__ == "__main__":
533         main()
534
```

7. assistant (2026-06-19T15:46:42.855Z)

Now let me read the eval docs in my slice.

8. user (2026-06-19T15:46:43.383Z)

```
1       # Annotation & Evaluation
2
3       How to measure how well the pipeline detects rallies, using hand-annotated
4       ground-truth timestamps. The harness scores predictions **already stored in the
5       SQLite DB** ? it makes no API calls and is fully deterministic, so you can run it
6       as often as you like for free.
7
8       ---
9
10      ## 1. What you annotate (and what it does / doesn't do)
11
12      You annotate, per video, the `[start, end)` time range of each **rally**. That's
13      all the evaluation harness needs.
14
15      > **Important:** these rally timestamps measure and tune **rally segmentation**
16      > ? i.e. *where* the pipeline says rallies begin and end. They do **not** train
17      > the shuttle detector (TrackNet/WASB), which learns from per-frame shuttle
18      > `(x, y)` coordinate labels, a different and much heavier annotation. Use rally
19      > timestamps as your **golden evaluation set**; only invest in frame-level
20      > coordinate labels if evaluation shows *detection* (not segmentation) is the
21      > bottleneck. See `docs/TRACKNET_WSL_SETUP.md`.
22
23      ## 2. Annotation file format (CSV)
24
25      One CSV per video, one rally per row:
26
27      ```csv
28      start,end
29      00:01:12,00:01:39
30      102.5,131.0
31      ```
32
33      - The `start,end` header is optional (auto-detected).
34      - Timestamps may be **decimal seconds** (`102.5`) or **colon-separated**
35        `MM:SS` / `HH:MM:SS` (`00:01:12`). Forms can be mixed.
36      - Blank lines and `#` comments are ignored.
37      - Malformed rows (bad timestamp, `start >= end`) fail loudly rather than being
38        silently skipped ? so labeling mistakes surface immediately.
39
40      Generate an empty template to fill in:
41
42      ```bash
43      python run_eval.py --scaffold --annotations rallies.csv
```

```
 ============================================================
  Rally evaluation ? video 'demo'
  Ground-truth rallies: 3 | IoU threshold: 0.5
 ============================================================
  Stage         Pred   TP   FP   FN   Prec    Rec     F1   mIoU  dStart    dEnd
 ------------------------------------------------------------
  candidates       3    3    0    0   1.00   1.00   1.00   1.00    0.0s    0.0s
  final            2    2    0    1   1.00   0.67   0.80   0.95    0.2s    0.5s
 ============================================================
  [final] missed rallies (false negatives): 1
    - 00:01:20?00:01:32
```

- **TP / FP / FN** ? true positives (matched), false positives (spurious predictions), false negatives (missed rallies).
- **Prec / Rec / F1** ? precision, recall, and their harmonic mean.
- **mIoU** ? mean temporal IoU over matched pairs (how tightly matched rallies overlap their ground truth).
- **dStart / dEnd** ? mean absolute boundary error in seconds (are rallies detected but trimmed in the wrong place?).

The key diagnostic: candidates vs. final

Scoring both stages tells you *where* to spend effort:

- **Candidates miss rallies the GT has** ? the **detector** is the bottleneck. Improve the shuttle model (e.g. fine-tune TrackNet/WASB with frame-level labels) or relax detection thresholds.
- **Candidates catch them but `final` is wrong/trimmed** ? the **segmentation / AI handoff** is the bottleneck. Tune thresholds (`velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration`) or the AI prompt ? no new labels needed.

```
106
107     In the example above, detection is perfect (candidates: recall 1.0) but the AI
108     handoff dropped one rally (final: recall 0.67) ? so the fix is in segmentation,
109     not detection.
110
111     ## 5. How matching works
112
113     Predictions and ground truth are sets of `[start, end)` intervals. A prediction
114     **matches** a ground-truth rally when their **temporal IoU** (overlap ÷ union) is
115     at least the threshold. Matching is **greedy and one-to-one** ? highest-IoU pairs
116     are claimed first, and each prediction/GT is used at most once, so a single
117     detection can't be credited for two rallies. Unmatched predictions are false
118     positives; unmatched ground-truth rallies are false negatives.
119
120     ## 6. Code map
121
122     | File | Role |
123     |------|------|
124     | `backend/eval/metrics.py` | Pure interval math: IoU, matching, P/R/F1, boundary error.
|
125     | `backend/eval/annotations.py` | Load/validate the ground-truth CSV. |
126     | `backend/eval/harness.py` | Read predictions from the DB, score, format the report. |
127     | `run_eval.py` | CLI entry point. |
128     | `tests/test_eval_*.py` | Deterministic tests for all of the above. |
129
130     ## 7. Troubleshooting
131
132     | Symptom | Cause & fix |
133     |---------|-------------|
134     | `ERROR: database not found at output/...` | The harness only reads stored predictions
? it does **not** run the pipeline. **Process the video first** (e.g.
`scripts/run_process_and_stitch.py`) so detections land in the DB, then re-run. |
135     | Every rally shows as a miss (FN), `Pred` is 0 | This video hasn't been processed, or
you used the wrong `video_id`. The pipeline keys videos by **file MD5** ? pass the *same* file
via `--video`, or the exact stored `--video-id`. |
136     | `video_id` doesn't match although it's "the same" video | Re-encoding/trimming changes
the bytes, hence the MD5. Use the exact file that was processed, or look up the stored id (it's
printed during processing). |
137     | `candidates` stage is empty but `final` has data (or vice-versa) | Depends on which
segmenter ran. `gemini` writes only `final` (no candidates); `yolo_hybrid`/`tracknet_hybrid`
write both. Use `--stage final` for the gemini segmenter, and `--detector <name>` to scope
candidates. |
138     | Recall looks low only at high `--iou` | Rallies *are* found but boundaries are loose ?
check `dStart`/`dEnd`. Tune `stitching` padding or the segmentation thresholds rather than the
detector. |
139     | `ValueError: start (...) >= end (...)` while loading annotations | A bad row in your
CSV. Timestamps must be increasing per rally and use seconds or `HH:MM:SS`. |
140
141     Run `python run_eval.py -h` for a quick reminder of this prerequisite and all flags.
142
```

9. user (2026-06-19T15:46:43.772Z)

```
1       # R2 ? Task-faithful rally-boundary eval metric (tolerance-match + over-seg guard)
2
3       **Status:** **IMPLEMENTED (augmenting), 2026-06-17** ? owner-reviewed + decisions locked
2026-06-17;
4       `backend/eval/tolerance_metrics.py` + `rally_seg_eval` reporting landed (default-
additive; tIoU stays the
5       gate pending the ablation experiment). Gates the rally-end work (R4) and the dual-eval-
set regression gate.
6       A focused increment of the rally-quality project
([RALLY_QUALITY_RESEARCH.md](RALLY_QUALITY_RESEARCH.md) §7;
7       the day-1 validation that motivates it:
[REAL_FOOTAGE_VALIDATION_REPORT.md](REAL_FOOTAGE_VALIDATION_REPORT.md);
```

the directive: `eval-boundary-tolerance` + `eval-dual-set-regression`).

> **Implementation refinement (2026-06-17):** §2.3.2's boundary-error distribution is computed over
> **unconditional best-overlap** matches, NOT the within-? 1:1 matches ? a ?-gated set is survivorship-biased
> (loose ends fail the gate and vanish, hiding the very end-deficit the diagnostic exists to surface; that
> within-? deficit instead shows up as low `tol_recall`). Matches the day-1 validation method. Measured on the
> n=9 corpus: **|?start| 1.25 s vs |?end| 2.19 s** ? "start?solved, end=the gap", confirmed.

---

0. TL;DR

`tIoU-F1@0.5` is the wrong ruler for rally detection, in **both** directions: it over-penalizes the
intrinsically fuzzy rally START (the ~1?1.5 s service window), and its overlap matching is murky about
over-production. R2 replaces the headline with a **tolerance-match metric under strict 1:1 (Hungarian)
assignment**, reported as a **decomposition** ? `P@? / R@? / F1@?` + **boundary-error distributions split
into start/end** + an **over-segmentation guard** + a **?-sweep** ? not a single number.

Honest, slightly uncomfortable consequence (measured, see §3): under R2 the local detector's headline
number *drops* and the gap to Gemini *widens* vs what tIoU showed ? because R2 faithfully charges for the
1.5× over-production and the loose ends that tIoU's overlap matching partly absorbed. That is the point:
R2 splits "behind Gemini" into its two real causes ? **(a) over-production [precision]** and **(b) loose
rally-ends [the R4 lever]** ? and forgives only the inherent start fuzziness. It is a more honest ruler,
not a flattering one.

---

1. Why `tIoU@0.5` is the wrong ruler (grounded in the n=2 ground-truth clips)

1. **Over-penalizes the START.** For a 5.7 s rally, `tIoU ? 0.5` tolerates only ~±1.9 s of boundary shift
   (`? = D·(1??)/(1+?)`). The service window (stance ? racquet contact) alone is ~1?1.5 s ? even **Gemini's**
   median |?start| is 0.98?1.46 s. So tIoU spends most of its budget on a boundary that is *intrinsically*
   undefined to ~1 s. (Owner directive: don't strictly align starts; tolerate ~±2 s.)
2. **Count parity is a deceptive proxy.** mahadevpura-2 hit 1.03× of Gemini's count yet F1@0.5 = 0.150 ?
   right count, wrong boundaries. A count- or overlap-based view hides this.
3. **It flips config rankings on noise.** The hard-cap was golden-neutral at n=6 (+0.006 n.s.) but "best" at
   n=7 once a continuously-active clip entered ? a metric artifact at the 0.5 cliff, not a quality change.
4. **The deficits we must see are start?end-symmetric.** Local's |?start| ? Gemini's; the **entire** gap is
   the rally-**END** (|?end| 2.1?3.1 s vs Gemini ~0.8?1.2 s). A single blended IoU cannot show that ? and
   directing R4 needs exactly that decomposition.

---

2. The metric

```
51      ### 2.1 Match rule (asymmetric tolerance ? forgive the start, expose the end)
52      A predicted window matches a golden rally iff **`|?start| ? ?_start` AND `|?end| ?
?_end`**.
53      - **`?_start` generous (~2.0 s)** ? absorbs the service-window/annotator start fuzziness
(the part that is
54        *not* a model deficit).
55      - **`?_end` tighter (~1.5 s, but IAA-calibrated ? see §2.4)** ? the rally-end IS a sharp
physical event
56        (shuttle down/net/out), so a loose end *is* a real deficit we want the metric to
expose, not forgive.
57
58      > A *symmetric* ±2 s tolerance is tempting (the owner's stated bar) but the preview (§3)
shows it is **not**
59      > automatically more lenient than tIoU 0.5 and it blurs exactly the end-deficit we need
to see. Asymmetry is
60      > the point: relax the dimension that's intrinsically fuzzy (start), keep honest the one
that's a real event (end).
61
62      ### 2.2 Assignment ? strict 1:1 (Hungarian), never overlap/greedy
63      Match references to predictions by **global Hungarian assignment** (max total matches
under the tolerance
64      gate), one-to-one. This is what makes **over-production cost precision directly**: `P@?
= matched / n_pred`
65      crashes when the detector emits 75 windows for 52 rallies. (tIoU's overlap matching let
extras "match
66      something"; the W1 over-segmentation pendulum was barely costed.)
67
68      ### 2.3 Report the DECOMPOSITION, never one number
69      1. **`P@?`, `R@?`, `F1@?`** ? precision (over-production), recall (did we find the rally
within tolerance), F1.
70      2. **Boundary-error distributions** ? median **+ IQR** of signed *and* absolute
`?start`, `?end` over matched
71         pairs, **split**. This is the metric that says *"start solved, end open"* and aims
R4.
72      3. **Over-seg guard (mandatory):** `split_count` (preds covering one ref ? W1 over-seg)
and `merge_count`
73         (refs under one pred ? the baseline mega-window). The *original* rule was strict per-
video: **a promotion
74         may not increase either on ANY video.** **REFINED to a NET guard, 2026-06-17 (R1,
owner-approved)** ? the
75         strict rule is correct for an *incremental* cue but **mis-fires on a structural mega-
window?bounded change**
76         (splitting a mega-window necessarily lifts `split_count` from ~0, and the raw merge
*count* is non-monotonic:
77         one window hiding 40 rallies is `count=1, rate=1.0`; bounding it into pieces that
each glue 2 neighbours is
78         `count=9, rate=0.5` ? FEWER rallies hidden). The net guard scores over-seg on the
**normalized rates** as one
79         weighted cost per video (`w_merge·merge_rate + w_split·split_rate`, merges weighted
higher ? a merge hides a
80         rally = recall loss, a split only fragments one), and **passes iff (a) the corpus-
mean net cost does not rise
81         AND (b) no video's `merge_rate` regresses.** Implemented as
`tolerance_metrics.over_seg_guard` (it always
82         reports the strict verdict too; revert = read `strict_passes`). See
QUALITY_ITERATIONS.md "R1" for the measured
83         verdict (milestone: mean `merge_rate` 0.694?0.150, net PASS, strict BLOCK).
84      4. **?-sweep** (`? ? {1.5, 2.0, 2.5, 3.0}`, symmetric for the trend; asymmetric pair for
the headline) ?
85         because the convention/IAA noise is large, a single ? misleads (§3). Report the
curve.
86      5. **`tIoU`/`mIoU` demoted to a SECONDARY trend-line diagnostic** (keeps the paper-
ablation backbone
87         comparable) ? **not the gate.**
88
```

2.4 Choosing ? ? it MUST be calibrated to inter-annotator agreement
The preview (§3) shows even **Gemini** scores `tolF1@1.5 = 0.20` on mp2 ? i.e. on ~half the rallies the
golden END differs from Gemini's by >1.5 s. That is plausibly **label/convention noise, not model error**:
we'd be measuring how two humans (or human-vs-Gemini) disagree on "when did the rally end." **Therefore
`?_end` must be ? the inter-annotator end-agreement**, which we do not yet know. **Action:** a small **IAA
study** ? a 2nd labeler re-labels 2?3 already-golden videos; set `?_start`/`?_end` to ~the 90th-percentile
pairwise human |?start|/|?end|. Until then **freeze provisional ?_start=2.0, ?_end=1.5 and always report the
sweep**, with the `? = D·(1??)/(1+?)` derivation recorded so the choice is auditable.

---

3. Preview on the two ground-truth clips (greedy-1:1 approximation; Hungarian in the impl)

Symmetric-? sweep, F1@? (local-best = the per-clip best config: mp2 W1+W2+hardcap, GX010128 W1+W2):

clip / method	tolF1@1.5	@2.0	@2.5	@3.0	(tIoU-F1@0.5)
mahadevpura-2 / **Gemini**	0.203	0.354	0.506	**0.557**	0.557
mahadevpura-2 / local	0.111	0.148	0.222	0.278	0.296
GX010128 / **Gemini**	0.615	0.769	0.813	**0.857**	0.813
GX010128 / local	0.189	0.315	0.346	0.362	0.457

Reading the preview (all honest):
- **`tolF1@~3.0 ? tIoU-F1@0.5`** ? the sweep is a smooth, interpretable knob that *contains* tIoU as a point;
  good (continuity with the frozen baseline).
- **The local?Gemini gap WIDENS under the tighter, 1:1 tolerance** (GX010128 ratio 0.56 at tIoU0.5 ?
  0.41 at ?2.0). R2 charges local for the 1.5× over-production (precision) and the loose ends ? exactly
  the deficits tIoU under-counted. **This is the metric working, not local regressing.**
- **Even Gemini drops at small ?** (mp2 0.557?0.203 from ?3.0?1.5) ? strong evidence the **golden END
  convention carries >1.5 s of noise** ? §2.4's IAA calibration is a prerequisite, not a nicety.
- Boundary-error medians (the robust diagnostic, from the validation report): local |?start| 0.78?1.74 s
  (? Gemini 0.98?1.46) vs local |?end| 2.12?3.12 s (? Gemini 0.76?1.19) ? **the headline R4 signal.**

---

4. Promotion / "trustworthiness" rule (small-n)
A candidate is promotable only if **all** hold:
1. **Paired per-video sign-test sweep on F1@?** is near-unanimous: **6/0 @ n=6 (p=0.031)** or **7/0 @ n=7
   (p=0.016)** (8/0 @ n=8, p=0.008); 5/1 or 6/1 is NOT significant. Run on the **growing-golden** set.
2. **No over-seg regression** ? the **net** over-seg guard passes (§2.3.3): the corpus-mean weighted
   merge/split-*rate* cost does not rise AND no video's `merge_rate` regresses. (Was the strict per-video
   count rule; refined to net on 2026-06-17 ? see §2.3.3.)
3. **Effect size + bootstrap CI** reported as an honesty band (expect it to span 0 at this n; use it to
   separate "real" from "directionally-clean-but-negligible," not as the pass/fail).
4. **Stratify, don't blend** ? report the continuously-active blob clip (mahadevpura-1)

as its own stratum
(its near-degenerate scores can swing a small-n mean).
5. **No regression on EITHER eval set** (raw + growing-golden) ? the dual-set discipline.

---

5. Implementation plan (pure, offline, additive ? zero production risk)
- **`backend/eval/tolerance_metrics.py`** (pure, GPU-free, unit-testable):
  - `match_1to1(preds, gts, ?_start, ?_end) -> (matches, P, R, F1)` ? Hungarian via `scipy.optimize.linear_sum_assignment` if available, else a pure O(n³) fallback (corpus n is tiny).
  - `boundary_error_report(matches) -> {?start, ?end: signed/abs median+IQR}`.
  - `over_seg_report(preds, gts) -> {split_count, merge_count}` (reuse the bipartite logic from `segmentation_metrics.merge_split_report`).
- **Wire into `backend/eval/rally_seg_eval.py`** ? emit the R2 block **alongside** tIoU (don't remove tIoU); add `--tau-start/--tau-end/--tau-sweep`. Surface in the experiment leaderboard.
- **`run_regression.py`** ? add the dual-set R2 aggregate + the over-seg-guard gate. *(Status: the guard primitive landed as `tolerance_metrics.over_seg_guard` and is wired into `rally_seg_eval.compare()`/`--vs` ? the natural home, since `rally_seg_eval` loads the golden manifest + trajectories. The dual-set `run_regression.py` aggregate is still a separate future item.)*
- **Tests** ? synthetic intervals: exact-match, start-fuzzy-within-?, end-loose-beyond-?, over-production (precision crash), the Hungarian-vs-greedy tie case, empty inputs.
- **Default-additive ? ablation-gated replacement:** R2 is reported next to tIoU (augment). The *gate* switches to R2 only after a dedicated **ablation experiment** on the golden corpus shows R2 ranks candidates at least as sensibly as tIoU ? so the promotion bar never changes silently mid-flight. When tIoU is retired as the gate it is **deprecated with documentation, not deleted** (kept as the secondary trend-line + the `QUALITY_ITERATIONS.md` paper-ablation backbone, with a dated note recording the ablation evidence for the switch). Owner decision, 2026-06-17 (§7).

---

6. Definition of done
- `tolerance_metrics.py` + tests merged (default-additive); `rally_seg_eval` reports the R2 decomposition + ?-sweep next to tIoU; the corpus (n?8) re-scored under R2 and recorded in QUALITY_ITERATIONS.
- The IAA study (§2.4) scoped (it can land after R2's code; R2 ships with provisional ? + the sweep).
- `run_regression.py` carries the dual-set R2 gate + the per-video over-seg guard ? but R2 **augments** tIoU until the ablation experiment (§7) validates it as the gate; only then does the gate switch.
- tIoU/mIoU retained as a secondary trend-line; when retired as the gate, **deprecated-with-docs (kept for posterity)** per the §7 owner decision ? a dated note records why and links the ablation evidence.

---

7. Decisions
Locked (recommended, design-level): Hungarian 1:1 (not greedy/overlap); report the decomposition + sweep, never one number; tIoU demoted to secondary; R2 default-additive until validated;

```
boundary-error split
177     start/end is the R4-aiming diagnostic.
178
179     **Locked (owner review, 2026-06-17):**
180     - **?_end asymmetric (1.5 s) vs ?_start (2.0 s) ? ADOPTED.** Asymmetry is the point:
forgive the intrinsically
181       fuzzy start, keep the rally-end honest. Your "±2 s, not more" bar maps to
`?_start=2.0`; `?_end` stays the
182       tighter, IAA-calibrated knob (§2.4).
183     - **IAA study after R2 code ? ADOPTED.** R2 ships now with provisional `?_start=2.0 /
?_end=1.5` + the full
184       sweep; the IAA study lands next to fix ?. R2's code is not blocked on it.
185     - **Gate: AUGMENT now, then ablation-gate the eventual replacement, then deprecate-with-
docs ? ADOPTED.**
186       R2 is reported **alongside** tIoU (augment) until validated. tIoU is **not** swapped
out silently: before
187       R2 becomes the promotion gate, run a dedicated **ablation experiment** on the golden
corpus showing R2
188       ranks candidates at least as sensibly as tIoU (and exposes the deficits tIoU hid).
When tIoU is finally
189       retired as the gate, it is **formally deprecated with documentation ? kept for
posterity, never deleted**:
190       it remains a secondary trend-line and the paper-ablation backbone
(`QUALITY_ITERATIONS.md`), with a dated
191       note recording *why* it was superseded and the ablation evidence behind the switch.
(Consistent with the
192       repo's "retain weak/superseded signals; numbers inform, foundations decide" norm.)
193
```

10. assistant (2026-06-19T15:46:46.764Z)

Now let me read the remaining three docs in my slice.

11. user (2026-06-19T15:46:47.914Z)

```
1       # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by
design)
2
3       **Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` +
`tests/test_experiment.py`, shipped
4       2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest),
`compare_unlabeled()`
5       (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**
Phases below are
6       sequenced but each is a separate PR.
7
8       > **Companion docs:** the cues this experiments *on* are in
9       > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10      > [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels +
`baseline.json` come from
11      > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is
**ADR-007**
12      > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled
classifier is
13      > **ADR-004**. The 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15      ---
16
17      ## 1. Context ? why this, why now
18
19      PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues,
all **default
20      OFF**. The owner's directive for *how* we evolve quality from here:
21
```

> *"Evolve the codebase so we can keep finding new ways to improve quality and experiment with them ?
> or add them as a signal ? without suffering any regressions if the idea doesn't work out. Rolling
> back the code would be too dangerous."*

So **experimentation and deployment must be decoupled.** The contract:

1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production behavior.
2. Test it **offline against golden labels** (most experiments never touch the served path at all).
3. Promote a variant to default **only on measured lift**, through a CI eval gate.
4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned variant that is
   always present.

The pleasant surprise (Explore audit, 2026-06-13): ~80% of this harness already exists and is
simply not composed *as a system*. The missing piece is a thin **variant/experiment layer + the
discipline**, not a platform. §4 is the exact code surface so the next session does not re-discover it.

---

2. The thesis ? separate expensive DETECTION from cheap DECISION

The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per frame" (shuttle
WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python part is the
decision: "given those signals, where are the rallies" (windowing + gates + the classifier).

> **Run detection once per video, persist all per-cue signals + per-candidate features, then re-score
> any number of variants OFFLINE ? deterministically, with no GPU and zero production risk.**

This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do exactly this on
stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence substrate.
? Most experiments are pure re-scoring of stored data; they never enter the served pipeline.

---

3. What already exists (the audit ? do NOT rebuild)

Exact public surface, with `file:line`. **This table is the anti-duplicate-work artifact** ? start here.

3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
`backend/eval/metrics.py`
- `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
- `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1 IoU matching.
- `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1, mean IoU, boundary errors; `Score.as_dict()`.
- `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).

3.2 A/B + cross-validation primitives (the comparison engine already exists)
`backend/eval/calibration.py`

- `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5) -> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
- `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113) ? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
- `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) -> dict` (:72) ? within-video k-fold overfit guard.
- `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config, float)` (:61).
- `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
- LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.

3.3 No-regression gate + baselines (the promotion gate already exists)

`backend/eval/regression.py`

- `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None, tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ? score stored preds vs GT, flag if P **or** R drops > tolerance.
- `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
- `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)` (:80) · `load_baselines(path) -> dict` (:68).
- `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R, `tolerance`, `has_baseline`; `as_dict()`.
- Default baseline file: `eval_baselines/regression_baseline.json` (:33).

`run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1 regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations --stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-missing-baseline --json`.

3.4 Pinnable model artifact

`backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2): `fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75), `predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON) (:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**

3.5 Offline re-scoring substrate (persist once, re-score forever)

`backend/infrastructure/database.py`

- `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None, confidence=None, stats=None, detection_params=None) -> Optional[int]` (:318) ? `stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`, `detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
- `query_candidate_segments(video_id, detector=None, only_unlabeled=False) -> List[dict]` (:355) ? returns rows with deserialized `stats`.

3.6 Label assignment + feature glue

`backend/eval/training.py`

- `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]` (:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
- `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5, feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.

3.7 Existing LOVO drivers to mirror (not duplicate)

- `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)` (:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`; `FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40); already does LOVO over Gemini silver labels.
- `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77); `LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
- `backend/eval/calibrate_wasb.py` ? `run(...)`, `load_golden_gts()`; `backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden reader.

3.8 Registries + config knobs (variant selection)

- `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton `segmenter_registry` (:63); registered (6): `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`.

- `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton `annotation_registry`; providers `file`, `auto`.
- `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). The `min_crossings` knob now lives on `FusionConfig` (default `0` = OFF; also `min_players`, both served Gate A/B thresholds ? see fusion P2).

3.9 Confirmed ABSENT (no duplication risk)

Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a stray "A/B test" aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note in `archives/research/AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical comparison mechanism.

---

4. The variant abstraction (the one thin thing to add)

A **`Variant` = a declarative recipe**, not a code branch:

```
Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params, classifier model path/hash)
```

JSON-serializable; selected via the existing `segmenter_registry` + `IndexingConfig.default_segmenter`.

Control = a *pinned, frozen* variant (recipe + model hash) that is always registered and is the default. Adding a new cue = a new Variant differing by one flag ? the control recipe is untouched.

---

5. Three experiment modes

5.1 Offline A/B (primary, label-driven, zero production risk)

`compare(videos, variant_A, variant_B)` ? a thin driver composing **existing** functions:

`query_candidate_segments()` (load persisted features) ? re-score each variant ? `calibration.improvement_report()` + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden (`gt_loader.load_gt_intervals`).

Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test, and it needs almost no new scoring code** ? only the glue.

Honest reporting (default ON, additive): `compare(..., honest_stats=True)` also attaches a `"honest"` block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and **F1@k + segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output unchanged; `honest_stats=False` restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the lift is real when the ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in ANALYZERS_AND_RECONCILERS.md §13/C1+C4 (helpers: `backend/eval/eval_stats.py`, `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the leaderboard.

5.2 Shadow (for cues that touch detection, e.g. the person detector)

```
147     The new cue runs and **persists its signal** into the candidate
`stats`/`detection_params`, but the
148     *served* decision stays control. Offline you then ask "what would variant B have
scored?" via §5.1 ?
149     B never affects output until it wins. This is how a detection-touching change (not just
a re-scoring
150     change) earns its way in without risk.
151
152     ### 5.3 Promotion gate (no-regression, CI-enforced)
153     `regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant
becomes the new
154     default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1
blocks). Control is
155     pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**
156
157     ---
158
159     ## 6. No-regression guarantees (the heart of the owner's ask)
160
161     - **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged
code can't change
162       behavior until explicitly enabled.
163     - **Control variant pinned** ? frozen recipe + classifier model hash, always registered
& default.
164     - **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no
silent default change.
165     - **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set
hash + timestamp,
166       persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`).
Answers "which
167       signals ever helped, on which video slices," and makes every result reproducible.
168     - **Decoupled events** ? "merge an experiment" and "change the default" are separate,
independently
169       revertible actions.
170
171     ---
172
173     ## 7. What to build later (gap list ? thin, additive, owner-gated)
174
175     1. ~~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)`
composing the §3
176        functions (no new scoring math).~~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned
`CONTROL`),
177        `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring
`distill_local`),
178        `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via
`match_intervals`,
179        no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and
`load_video_candidates`
180        (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a
`VariantRegistry`
181        only if the count grows. 19 tests incl. the no-regression invariant.
182     2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
`two_sided_motion`,
183        `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
`direction_reversals`;
184        audio: hit times) so any future variant re-scores offline ? ties to
`MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
185     3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter`
already exists.
186        Add the `min_crossings` knob (fusion P2) here too.
187     4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`;
write the
188        experiment-leaderboard JSON.
189
```

```
190     ---
191
192     ## 8. Phasing (owner-gated; ONE PR per slice, off `master`)
193
194     - **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from
`MULTI_SIGNAL_FUSION_PLAN.md` and
195       `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
196     - **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over
`calibration` +
197       `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container
(`compare`,
198       `compare_unlabeled`, `record_experiment`, the no-regression invariant).
`compare_unlabeled` adds the
199       SxS read on **new unlabeled footage** the owner asked for (run control + candidate on
a fresh video,
200       diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-
side follow-on).
201     - **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default
OFF) ? couples
202       with fusion P1/P2.
203     - **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
204
205     ---
206
207     ## 9. Scope guardrail
208
209     **Single-tenant / local-first** (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment
registry + the
210     existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of
`baseline.json`.
211
212     ---
213
214     ## 10. Verification
215
216     - **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit
test that
217       `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant
test** asserting
218       "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test
asserting
219       `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
220     - **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run
`compare(control,
221       fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-
check.
222     - **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
223       `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
224     </content>
225
```

12. user (2026-06-19T15:46:48.115Z)

```
1       # R1 Milestone ? Launch Report (windowing default flip: W1 cap=6 + R4)
2
3       **Status:** LAUNCHED 2026-06-18 (PR #204). **Flip:** the local-windowing defaults go ON
?
4       `max_window_duration 0?6`, `window_inactivity_end 0?1.5`, `inactivity_rally_thresh
0.08?0.20`,
5       `inactivity_min_density 0.34?0.30`. **W2 net-crossings, W1.5 hard-cap, and R5 blob-
fallback all stay OFF.**
6       This is the [Launch Report convention](../../) record made **before** the flip merged ?
total quality impact
7       on **both rulers** + the over-seg guard + honest caveats, for posterity.
```

> **Why this flip is safe to ship:** measured on the **n=13** golden corpus (the grown set, incl. the two
> Boxhill clips GX010137/GX010141), the milestone clears the **net over-seg promotion bar** with room to
> spare, and the verdict was **independently reproduced** (a different harness, #214) and **adversarially
> stress-tested** (5-lens LOO/seed/per-clip workflow). Nothing in the verification refuted it.

---

1. Total quality impact ? BOTH rulers (n=13, leave-one-video-out unit = whole video)

The headline promotion metric is the **R2 task-faithful tolerance-match F1** (`tolF1`, asymmetric ?_start=2/
?_end=1.5); **tIoU-F1@0.5** is kept as the secondary continuity ruler.

config	**R2 tolF1**	**tIoU-F1@0.5**	mean |?end|
baseline (all-OFF)	0.137	0.236	44.6 s
+ W1 cap=12	0.210	0.382	5.7 s
+ cap=6 (R3)	0.320	0.436	5.0 s
+ R4 (milestone, shipped)	**0.359**	**0.474**	**3.3 s**

Paired sign-tests (the promotion evidence):

transition	? tolF1 (95% CI)	sign (W/L/T)	p	? tIoU-F1	sign	p
baseline ? milestone	**+0.222** [+0.152, +0.286]	**12/0/1**	**0.0005**	+0.238 [+0.159, +0.318]	12/0/1	0.0005
cap12 ? cap6 (R3)	+0.110 [+0.070, +0.153]	12/0/1	0.0005	?	?	?
cap6 ? +R4	+0.040 [+0.015, +0.065]	9/1/3	0.0215	?	?	?

The single non-win in baseline?milestone is **mahadevpura_1** (the continuously-active blob: tolF1 0.000?0.000,
a tie ? it needs R5, which stays OFF). **Both new clips improve:** GX010137 0.448?0.685, GX010141 0.333?0.431.

2. Over-segmentation guard (the promotion gate)

The R1 **net** over-seg guard (`tolerance_metrics.over_seg_guard`) is the gate (not the strict per-video count
rule). Baseline ? milestone:

- **NET verdict: PASS** ? weighted (merge:split = 2:1) merge_rate/split_rate cost **1.303 ? 0.429** (? ?0.874).
- **mean merge_rate 0.651 ? 0.161** ? drops on **all 13** videos; **zero per-video merge_rate regressions**.
- **strict_passes = False BY DESIGN** (not a regression): the raw per-video *count* rule trips because splitting
  a mega-window lifts split_count from ~0 and the raw merge *count* is non-monotonic (one window hiding 40
  rallies = count 1/rate 1.0 ? bounded into pieces gluing 2 neighbours = count 9/rate 0.5 ? **fewer** rallies
  hidden). The guard scores the recall-meaningful *rate*; see QUALITY_ITERATIONS.md "R1".

3. Verification provenance (independent + adversarial)

- **Independent reproduction (#214 `lovo_resweep.py`, a different harness):** milestone vs baseline 0.137?0.359,
  sign **12-0-1, p<0.001**; R4 on-vs-off 0.320?0.359, sign **9-1-3, p=0.021** ? matches to 3 decimals. Its LOVO
  cap-selection picks **cap=6 unanimously (13/13)** on tIoU.

```
56      - **5-lens adversarial workflow** (reproduce / leave-one-out / R4-loss audit / new-clip
sanity / guard audit ?
57        synthesis): verdict **STANDS**. Every LOO fold (n=12) clears CI-excludes-0 AND p<0.05;
worst fold (drop
58        GX010128, the biggest gainer) still ?+0.202, p=0.0010 ? **no single clip drives the
win**.
59
60      ## 4. Honest caveats (recorded so they aren't relitigated)
61
62      1. **The W1 cap is the dominant lever; R4 is a marginal increment.** baseline?cap6 is
~+0.18 of the +0.222
63         tolF1 gain; the R4 inactivity-end trim adds a *small but significant* +0.040
(p=0.0215, sign 9/1/3). Frame R4
64         as "a small, significant end-precision gain" (mean |?end| 4.96?3.31 s), not a co-
equal driver.
65      2. **R4's one loss is GX010137** (tolF1 0.727?0.685, ?0.042) ? a tolerance-accounting
artifact, **not** an R4
66         failure: on that clip strict F1 *improves* (0.805?0.822), |?end| *improves*
(0.447?0.439), over-seg unchanged;
67         R4 simply trimmed 4 trailing predicted segments (43?39, toward gt=34). The clip's
*overall* milestone delta is
68         a large win (+0.237).
69      3. **GX010141 is the worst-behaved clip post-milestone** (merge_rate still 0.448, ~30 s
windows vs gt max ~10.5 s)
70         ? partially-merged, **not** a regression (it improves +0.098) and it does not break
the guard, but it's a
71         future over-merge target as the corpus grows.
72      4. **cap=4-vs-6 wobble on tolF1** (from #214): cap=4 edges cap=6 by ~0.005 (noise) ? but
**LOVO re-fitting to
73         cap=4 generalizes *worse*** (??0.004) and tIoU is firmly cap=6, so **cap=6 stands**.
Watch as n grows; switch
74         only if cap=4 pulls ahead *consistently across folds*.
75      5. **n=13 is small.** "Robust" here means "no single clip or bootstrap seed flips the
verdict," not large-sample
76         certainty. (Bootstrap CIs jitter ~0.001?0.004 run-to-run; means/sign-tests/p-values
are exact.)
77      6. **Nightly re-baselined + an all-OFF floor added (DONE).** The flip made the nightly's
`default` config
78         (= SERVED) equal `milestone`, so its config-drift detector flagged the frozen
baseline STALE ? re-snapshotted
79         in #216. A pinned **`baseline_off`** reference config was then added (follow-up) so
every nightly report keeps
80         showing the delta the milestone buys (floor tolF1 **0.137** vs milestone **0.359**) ?
the launched win has to
81         keep standing its ground, not be taken at launch-time value.
82      7. **Pure-logic eval only** (CPU, persisted trajectories) ? no GPU/real-video re-
detection was run; the flip
83         changes only the windowing knobs the served path already reads.
84
85      ## 5. What did NOT flip (still default-OFF, retained)
86
87      - **W2 net-crossings gate** ? no ground-truth-backed lift (the weak-signals-retained
directive keeps it).
88      - **W1.5 hard-cap** ? over-segments normal clips (chops before R4); superseded by R5 for
the blob case.
89      - **R5 blob-fallback** ? a stratum tool for continuously-active clips; turn on
selectively, not globally.
90
```

13. assistant (2026-06-19T15:46:51.826Z)

Now I have a thorough picture. Let me cross-check specific claims. The EXPERIMENT_HARNESS.md §3.8 cites specific line numbers in `models.py` ? let me verify those, and also confirm the `--tau-sweep` flag claim in R2 design doc.

14. user (2026-06-19T15:46:53.381Z)

```
No matches found
```

15. user (2026-06-19T15:46:54.551Z)

```
118:    default_segmenter: str = "yolo_hybrid"
121:    min_segment_duration: float = 3.0
125:    detector_model: str = "fasterrcnn_resnet50_fpn"
127:    yolo_velocity_threshold: float = 0.15
194:    @field_validator("default_segmenter")
```

16. assistant (2026-06-19T15:46:59.728Z)

Now let me check the EXPERIMENT_HARNESS §3.8 line-number citations more carefully. It claims `default_segmenter` at `:65`, but it's at line 118. However, line-number drift in a code-map can be defensible if the doc is a snapshot. Let me check the registered segmenters list claim (it says 6 registered) and the `min_crossings` location. Let me verify the segmenter registry and the EVALUATION.md matching claim against `metrics.py`.

17. user (2026-06-19T15:47:00.646Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
28          union = (a_end - a_start) + (b_end - b_start) - inter
29          return inter / union if union > 0 else 0.0
30
31
32      @dataclass
33      class Match:
34          """One matched predicted?ground-truth pair."""
35          pred_index: int
36          gt_index: int
37          iou: float
38
39
40      @dataclass
41      class MatchResult:
42          """Outcome of matching predictions to ground truth at a given IoU threshold."""
```

```
43          matches: List[Match] = field(default_factory=list)
44          unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
45          unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46
47
48      def match_intervals(preds: List[Interval], gts: List[Interval],
49                          iou_threshold: float = 0.5) -> MatchResult:
50          """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52          Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53          are claimed highest-IoU first, and each prediction/GT can be used once. This
54          keeps a single prediction from being credited for two ground-truth rallies
55          (and vice versa).
56          """
57          candidates = []
58          for pi, p in enumerate(preds):
59              for gi, g in enumerate(gts):
60                  iou = interval_iou(p, g)
```

18. user (2026-06-19T15:47:01.875Z)

```
backend\pipeline\segmenters\yolo_hybrid.py:65:        sport_name = self.config.get("sport",
"badminton")
backend\pipeline\segmenters\yolo_hybrid.py:74:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend\pipeline\segmenters\yolo_hybrid.py:75:        model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend\pipeline\segmenters\yolo_hybrid.py:375:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend\pipeline\segmenters\yolo_hybrid.py:376:        model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend\pipeline\segmenters\yolo_hybrid.py:436:segmenter_registry.register(HybridYoloSegmenter)
backend\pipeline\segmenters\wasb_hybrid.py:41:segmenter_registry.register(WasbHybridSegmenter)
backend\pipeline\segmenters\trajectory_hybrid.py:18:This base is deliberately NOT registered;
only concrete subclasses register.
backend\pipeline\segmenters\trajectory_hybrid.py:114:        sport_name =
self.config.get("sport", "badminton")
backend\pipeline\segmenters\trajectory_hybrid.py:128:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend\pipeline\segmenters\trajectory_hybrid.py:130:            model_name = "local-noai"
backend\pipeline\segmenters\trajectory_hybrid.py:135:            model_name =
self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend\pipeline\segmenters\motion.py:255:segmenter_registry.register(MotionPlaySegmenter)
backend\pipeline\segmenters\tracknet_hybrid.py:42:segmenter_registry.register(TrackNetHybridSegm
enter)
backend\pipeline\segmenters\fusion_hybrid.py:176:segmenter_registry.register(FusionSegmenter)
backend\pipeline\segmenters\base.py:39:    def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
backend\pipeline\segmenters\base.py:43:            name = temp_inst.name
backend\pipeline\segmenters\base.py:52:        """Returns a dict of registered segmenter names
and their descriptions."""
backend\pipeline\segmenters\gemini.py:74:        sport_name = self.config.get("sport",
"badminton")
backend\pipeline\segmenters\gemini.py:83:        provider_name = self.config.get("ai_provider",
idx_cfg.get("ai_provider", "gemini"))
backend\pipeline\segmenters\gemini.py:505:        model_name = self.config.get("ai_model",
self.config.get("indexing", {}).get("ai_model", "gemini-2.5-flash"))
backend\pipeline\segmenters\gemini.py:559:segmenter_registry.register(GeminiPlaySegmenter)
```

19. assistant (2026-06-19T15:47:13.161Z)

Now let me focus on the substantive staleness items. I've found several candidates. Let me verify the R2 doc's claim about the IoU sweep flag and the EVALUATION.md doc more carefully.

The key substantive contradictions to verify:

1. **R2_EVAL_METRIC_DESIGN.md §5** claims to "add `--tau-start/--tau-end/--tau-sweep`" ? but the CLI has `--tau-start` and `--tau-end`, NOT `--tau-sweep` (the sweep is computed by `tolerance_sweep` but no CLI flag exists). Let me verify there's no `--tau-sweep` anywhere.

2. **R2_EVAL_METRIC_DESIGN.md §2.3** references "§2.3.3" for the over-seg guard, but the section is numbered §2.3 item 3, not §2.3.3 ? this is a self-reference, not a code contradiction, so I'll skip it.

Let me check the milestone default values claim. The R1_MILESTONE doc says `inactivity_rally_thresh 0.08?0.20` and `inactivity_min_density 0.34?0.30`. In `models.py`: `inactivity_rally_thresh: float = 0.20` (line 154) and `inactivity_min_density: float = 0.30` (line 155). That matches. But `windowing.py` line 71-72 shows the `WindowingPreset` dataclass DEFAULTS are still `0.08` and `0.34` (the pre-flip values) ? but `_served_preset_from_defaults()` lifts the live config values, so SERVED gets 0.20/0.30. That's consistent.

Let me verify the EVALUATION.md harness path is still valid (run_eval.py, backend/eval/harness.py exist) and the claim about matching in `tolerance_metrics`. Let me also double check whether `--tau-sweep` exists anywhere and the IoU default.

20. user (2026-06-19T15:47:15.165Z)

```
56:    p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end per
row).")
57:    p.add_argument("--video", help="Path to the video file (its MD5 becomes the video_id).")
58:    p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
--video.")
59:    p.add_argument("--db", default=None, help="SQLite DB path (default: from config.json).")
60:    p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
62:    p.add_argument("--detector", default=None,
64:    p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
65:    p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep (0.1..0.9)
in the report.")
66:    p.add_argument("--show-failures", action="store_true",
68:    p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to this
path.")
69:    p.add_argument("--scaffold", action="store_true",
```

21. user (2026-06-19T15:47:15.715Z)

```
backend\eval\harness.py
```

22. user (2026-06-19T15:47:16.258Z)

```
Found 1 file
docs\R2_EVAL_METRIC_DESIGN.md
```

23. assistant (2026-06-19T15:47:25.595Z)

The `--tau-sweep` flag is mentioned only in R2_EVAL_METRIC_DESIGN.md, confirming it doesn't exist in code. However, §5 is the "Implementation plan" describing intended future work ? but the doc's own status header says "IMPLEMENTED" and the bullet says "Wire into `backend/eval/rally_seg_eval.py` ... add `--tau-start/--tau-end/--tau-sweep`". Since `--tau-start`/`--tau-end` ARE implemented but `--tau-sweep` is NOT, and the section is framed as completed implementation, this is a minor inaccuracy. But §5 is explicitly an "Implementation plan" ? I'll be conservative and not flag it as it's a plan section describing intent (the sweep IS computed by `tolerance_sweep`, just not exposed as a CLI flag). Let me focus on clearer contradictions.

Let me now focus on the EVALUATION.md doc ? it describes the legacy `run_eval.py` harness which is real, but I should check whether it omits the served operating point. The task hint

specifically says "served operating point described as pre-flip" and "wrong default values". Let me check QUALITY_ITERATIONS and RALLY_QUALITY_RESEARCH for the milestone defaults, and verify the windowing docstring claim about default values.

The key thing in `windowing.py`: the module docstring (lines 4-8) describes SERVED as `vt=0.02, merge_gap=2.0, min_window=2.0` ? but does NOT mention the W1 cap=6 / R4 defaults that are now ON. That's a code docstring, though, not in my doc slice. Let me check RALLY_QUALITY_RESEARCH.md §6 which the task points to.

24. user (2026-06-19T15:47:26.834Z)

```
1       # Rally-Detection Quality: Research, Eval Framework & Project Plan
2
3       **Status:** Foundation doc (2026-06-16), run as a **tracked project** (§7) ? *"Bridge
the
4       local?Gemini rally-detection gap,"* **completed when all committed tiers land and a
single quality
5       number is attributed (§7.1).** The empirical + research backbone for closing the gap
between the
6       **local, no-cloud rally segmenter** and the **Gemini** path. Iterations land as their
own measured
7       PRs, logged in [`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md); this doc is the plan
they execute against.
8
9       **How to read this:** §1 frames the gap with measured numbers. §2 is the precise, code-
grounded
10      diagnosis (read before proposing fixes). §3 inventories the eval/experiment harness we
already
11      have. §4 specifies how we strengthen that harness (the "really strong evaluation
framework").
12      §5 is the cited external-research synthesis (4 dimensions). §6 is the prioritized
roadmap with
13      falsifiable success criteria. **§7 is the project plan ? work-item tracker, sequencing,
the single
14      number we attribute to the effort, and the definition of done.** §8 recaps the non-
negotiable
15      constraints + cross-links.
16
17      > **This doc does NOT duplicate prior design.** It builds on, and cross-links:
18      > [`HOW_RALLY_DETECTION_WORKS.md`](HOW_RALLY_DETECTION_WORKS.md) (the 2-layer mental
model),
19      > [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) (the A/B + LOVO harness, P1 shipped),
20      > [`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) (shuttle+person+audio
fusion),
21      > [`ANALYZERS_AND_RECONCILERS.md`](ANALYZERS_AND_RECONCILERS.md) (the signal-fusion
framework +
22      > §13 literature review ? **already** establishes that our skeleton is standard TAL/TAS
+
23      > late-fusion; we *cite, don't re-derive*), [`ABLATION_LAYER.md`](ABLATION_LAYER.md),
24      > [`EVALUATION.md`](EVALUATION.md),
[`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md)
25      > (TAL-head-first ? VLM), and [`GOLDEN_SET_VENDORING.md`](GOLDEN_SET_VENDORING.md)
(corpus growth).
26
27      ---
28
29      ## 1. The gap (measured)
30
31      Two paths produce rally segments today. **Gemini** (cloud VLM, watches the video
semantically)
32      is strong; the **local** path (WASB shuttle trajectory ? velocity windowing, no cloud)
lags
33      badly. Measured F1 vs golden labels ([`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md)):
34
35      | Path | Mahadevpura (clean, single court) | Kushagra (multi-court) |
```

|---|---|---|
Heuristic velocity windowing (local today)	0.657	**0.157**
Gen-0 owned head (LogReg on 5 trajectory features, LOVO n=6)	0.744	0.561
Two-stage WASB + Gemini refine	0.83	?
Gemini wrapper (gemini-flash)	**0.93**	**0.66**

The local heuristic is **footage-fragile**: fine on a clean single court, collapses on multi-court / busy venues. The owned Gen-0 head already closes most of the multi-court gap (0.561 vs 0.66) ? the residual is **corpus-sized, not architecture-sized**.

1.1 The 2026-06-16 local-vs-Gemini divergence run (no golden labels)

A fully-local run (`wasb_hybrid --skip-ai-handoff`) vs Gemini on 3 fresh matches (Kushagra/Yash June-12; report: `?/June 12 Local-vs-Gemini/_LOCAL_vs_GEMINI_REPORT.md`, generator `scratch/compare_local_vs_gemini.py`). **Gemini is a strong reference here, NOT ground truth** (these clips are unlabeled) ? so this is an agreement/divergence study:

	Gemini	Local (WASB no-AI)
rallies / windows (3 videos)	59	12
total "play"	6:28	**17:41**
mean window duration	**6.1 s**	**65 s** (10.6×)
true misses (gemini-only)	?	**0**
merge groups (one local window ? ?2 Gemini rallies)	?	7

Headline: **the local path does not *miss* rallies ? it fails to *segment* them.** One mahadevpura-1 window spanned the entire 174 s clip. Local "play" is 2.7× Gemini's because its mega-windows swallow the dead time between rallies.

---

2. Diagnosis (code-grounded ? read before proposing fixes)

The local pipeline is **two layers** (see [`HOW_RALLY_DETECTION_WORKS.md`](HOW_RALLY_DETECTION_WORKS.md)): **(L1) detection** ? per-frame shuttle position (WASB/TrackNet); and **(L2) segmentation** ? turn the trajectory into rally intervals. The gap is almost entirely **L2**.

2.1 L1 (shuttle tracking) is NOT the bottleneck

On the divergence clips WASB reported the shuttle visible in **96?99 % of frames** (mahadevpura-1 98.6 %, GX020125 72.8 %, mahadevpura-2 78.8 %). The trajectory signal is present and dense.

2.2 L2 (segmentation) is the bottleneck ? and the flaw is conceptual

The windowing is `trajectory_to_action_windows` ([`backend/pipeline/detectors/tracknet_runner.py:256`](../backend/pipeline/detectors/tracknet_runner.py)):

- A frame is **"active"** iff the shuttle is **visible** AND its **per-second-normalized speed**
  exceeds `velocity_threshold` (default 0.02). Velocity = `(dist / frame_width) * (fps / dframes)`
  ? **already fps-normalized** (? fraction of frame-width per second).
- Active frames within `dead_time_merge_gap` (2.0 s) are **merged** into one window.
- Windows shorter than `min_action_window_duration` (2.0 s) are dropped.

The conceptual flaw: "the shuttle is moving" is treated as "a rally is in progress." During dead time the shuttle is still visible and gently moving (being picked up, walked back, knock-ups,

slow serve prep) ? those frames read as *active*. The 2 s merge then bridges the brief gaps, and
because there is **no `max_window_duration` cap and no inactivity/boundary split**, a single
window grows until the shuttle finally goes invisible ? which, at 96?99 % visibility, is rarely.
Result: rally + dead-time + rally + ? collapse into a few mega-windows.

> **Correction to an earlier hypothesis (honesty note).** A first pass guessed the cause was
> "velocity threshold not fps-invariant." It IS fps-invariant (the `fps/dframes` factor). The
> real cause is the *motion-equals-rally* conflation + 2 s bridging + no upper bound. fps matters
> only secondarily: denser 60 fps sampling keeps more slow-motion dead-time frames above the
> speed threshold. The `_LOCAL_vs_GEMINI_REPORT.md` has been corrected to match.

2.3 What this implies
The local path **has no rally-STATE signal** ? only "is the shuttle moving." Bridging the gap is
fundamentally about giving L2 signals that separate *rally* from *shuttle-merely-moving*
(net-crossings, two-sided play, sustained fast exchange, hit sounds) and a segmenter that
produces *bounded, well-cut* intervals ? exactly the multi-signal-fusion + learned-segmenter
direction the existing docs lay out. This doc makes that concrete and measurable.

---

3. What we already have (eval/experiment harness inventory)

The harness is rich and composable ? we **extend** it, not reinvent it. (Modules under
`backend/eval/`.)

Scoring & matching
- `metrics.py` ? temporal-IoU interval matching; precision/recall/F1, mean_iou, boundary errors.
- `segmentation_metrics.py` ? **F1@{0.1,0.25,0.5}**, **mAP@tIoU** (0.3?0.7), **segment_count_ratio**
  (the over/under-segmentation diagnostic). (Edit/segmental score deliberately omitted to date.)

Statistical rigor (small-n)
- `eval_stats.py` ? bootstrap 95 % CIs, paired exact **sign-test**, `paired_delta_ci`,
  `grouped_folds` (LOGO), `out_of_fold_predictions` (anti-stacking-leakage). The promotion bar =
  "?F1 CI excludes 0 **AND** sign-test agrees."
- `score_calibration.py` ? Platt + isotonic calibrators, **Brier + ECE**.

Variants, A/B, leaderboard
- `experiment.py` ? `Variant` (declarative recipe: segmenter + config + cue_flags + gates +
  learned-classifier + threshold/merge_gap/calibrate), `compare()` (LOVO-honest A/B with an
  "honest" stats block), `compare_unlabeled()` (agreement on unlabeled footage), `CONTROL`
  (pinned baseline), `record_experiment()` ? `eval_baselines/experiment_leaderboard.json`.
- `ablation.py` ? leave-one-signal-out marginal ?F1 with CI + sign-test + a no-over-claim verdict
  vocabulary (`earns_keep` / `suggestive` / `inert` / `structural_protected`).
- `classifier.py` ? dependency-free numpy `LogisticRegression` (the learned scorer; JSON-serializable).

Eval?serve discipline + gates
- `windowing.py` ? single-sources the **SERVED** preset from `IndexingConfig` defaults so eval
  and production can't silently diverge; `PROPOSAL` (recall-oriented) is where the golden feature
  substrate is built. (This exists *because* Gate-A measured **+0.074 on PROPOSAL but ?0.008 on
  SERVED** and was reverted ? the canonical eval?serve lesson.)
- `promotion.py` / `served_gate_a.py` ? promote a cue only if it does NOT regress at the served
  windowing; `per_user_gate.py` / `per_user_regression.py` ? per-user analogues; `regression.py` ?
  golden-set no-regression baseline with a GT-hash to detect label changes.

Golden substrate
- `gt_loader.py` ? one canonical GT loader (both legacy formats). `fusion_golden.py` /
  `fusion_compare.py` / `fusion_audio.py` ? extract replayable per-window feature JSONs
  (shuttle 5 + person 5 + audio 3) so ablation/compare re-score on CPU without re-detecting.
- **Corpus today: 6 golden videos** (label-set hash `50d98ffbc370`); ~10 more expected via the
  Roora vendoring track ([`GOLDEN_SET_VENDORING.md`](GOLDEN_SET_VENDORING.md)).

Signals available to L2 (built, default-OFF): net-crossing count (`rally_gate.py`, Gate-A),
5 person/court features (`person_detector.py` + `fusion_features.py`), 3 audio-hit features
(`audio_features.py`). Fusion hooks: `fusion_hybrid.py::_enrich_candidate_stats` /
`_select_for_handoff`. Persisted in `candidate_segments.stats`.

3.1 Known gaps in the harness (what §4 fills)
1. **Over-segmentation is under-surfaced in the default report.** `segment_count_ratio` exists but
   the merge/split *breakdown* (the very thing that exposed the over-merge) lives only in a scratch
   script. tIoU-F1 alone is **blind to over-merge** ? a giant window can still "match" via overlap.
2. **No segmental edit-score** (Levenshtein over the predicted/gt label sequence) ? the standard
   TAS over-segmentation metric.
3. **Per-cue calibration is fitted but never *measured* post-hoc** (no ECE/Brier delta reported).
4. **`out_of_fold_predictions` (anti-leakage stacking) is built but not wired to an arbiter.**
5. **No active-learning loop** ? nothing tells us *which* unlabeled video to label next.
6. **No single "rally-segmentation eval" entrypoint** ? the pieces exist but a one-command
   `score this segmenter vs golden, honestly` runner would make every iteration cheap.
7. **The local-vs-Gemini agreement track is ad-hoc** (a scratch script), not a first-class
   shadow signal alongside the golden gate.

---

4. The strengthened evaluation framework (the "really strong harness")

Design principles (inherited, non-negotiable):
- **Signals ? learned scorer, NO special-casing** ([`ANALYZERS_AND_RECONCILERS.md`](ANALYZERS_AND_RECONCILERS.md) §1):
  every new cue is a feature column the scorer learns to weight; never an `if/else` in the decision path.
- **Promote only on measured lift; default OFF; revert is one flag** ([`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) §6).
- **eval ? serve:** a harness win must be re-confirmed on the *served* windowing before any default flip.

- **Honest small-n stats:** never a bare mean at n?6?16; always CI + paired sign-test (C1).

4.1 Metric set the leaderboard should standardize on
Surface ALL of these in the default `compare()` report (most already computed; the task is to
report them together so over-merge can never hide again):

- **tIoU-F1 @0.5** (primary) **and F1@{0.1,0.25,0.5}** (segmental, TAS-standard).
- **mAP@tIoU** (0.3?0.7) when predictions carry confidences.
- **`segment_count_ratio`** + a **merge-rate / split-rate** breakdown (fraction of GT rallies that
  share a predicted window = under-segmentation; fraction of predicted windows that span ?2 GT =
  the over-merge signal). Promote the scratch `compare_local_vs_gemini.py` matching into a reusable
  `backend/eval/` helper.
- **Segmental edit (Levenshtein) score** ? the canonical over-segmentation metric we currently lack.
- **Boundary timing** (mean start/end error on matched pairs) ? already in `metrics.py`, surface it.
- **Calibration (ECE/Brier)** reported pre/post per-cue calibration (close gap #3).

> **Why:** plain tIoU-F1 rewarded our mega-windows (they "overlap" real rallies). The metric set
> above makes over/under-segmentation *first-class*, so a fix that improves segmentation is
> visible even when tIoU-F1 barely moves ? and vice-versa.

4.2 Splits & significance
- **Leave-one-video-out, grouped by match** (never leave-one-*window*-out ? windows within a video
  are correlated). Keep `grouped_folds`. Note the **RLOOCV bias** caveat for tiny n (see §5.4).
- **Bootstrap CI + paired exact sign-test** on per-video held-out F1 (C1). A result is *promotable*
  only if CI excludes 0 **and** the sign-test agrees, **and** it survives the served-windowing check.
- **Anti-leakage:** per-LOVO-fold threshold tuning + calibration on the TRAIN fold only (already
  enforced in `experiment._calibrate_and_tune`); wire `out_of_fold_predictions` once an arbiter exists.

4.3 The golden corpus + active learning (the highest-leverage lever)
The single biggest quality lever is **more labeled matches** ([`NEXT_STEPS.md`](NEXT_STEPS.md):
"corpus growth = highest leverage"). With ~6 ? ~16 videos:
- Grow via the Roora vendoring flow ([`GOLDEN_SET_VENDORING.md`](GOLDEN_SET_VENDORING.md),
  [`ROORA_LABELING_GUIDELINES.md`](ROORA_LABELING_GUIDELINES.md)) ? court calibration, IAA, the
  3 non-negotiables (ignore dead time, every rally, one-contact = one shot).
- **Active learning:** stop labeling videos at random. Score the *unlabeled* corpus and label the
  videos where the local segmenter is most *uncertain* / most *divergent from Gemini* / most
  diverse (new venue, fps, lighting). §5.3 surveys the methods; the local-vs-Gemini agreement
  signal (§4.4) is a ready proxy for "where do we disagree most ? label that."

4.4 Shadow evaluation: Gemini-agreement as a free, unlimited signal (NOT a gate, NOT training data)
We have exactly one strong reference available on *unlabeled* footage at inference time: **Gemini**.
- `experiment.compare_unlabeled()` already scores agreement (agreed / a-only / b-only)

without labels.
    - Use it as a **shadow metric**: run any candidate segmenter on a large unlabeled pool, measure
      agreement-with-Gemini, and watch it as a *trend* ? cheap, unlimited, catches regressions the 6
      golden videos can't. **It is a shadow signal only.** Golden labels remain the gate; Gemini is not
      ground truth (it has its own errors) and ? per the licensing guardrail (§8) ? **its outputs may
      never become training data for owned weights.**

    ### 4.5 One-command rally-segmentation eval (close gap #6)
    Wrap the above into a single reusable entrypoint (a future PR ? not built here): given a segmenter
    variant + the golden manifest, emit the full §4.1 metric set + the honest stats block + a leaderboard
    row, and (optionally) the §4.4 shadow agreement on an unlabeled pool. Every iteration in §6 is then
    "run the entrypoint, read the honest delta, promote-or-not."

    ## 5. External research synthesis (cited, adversarially verified)

    > Produced by a deep-research run (6 search angles ? 27 sources ? 126 candidate claims ? 25
    > 3-vote-adversarially-verified, 23 confirmed / 2 refuted). **Cite, don't over-claim.** Two claims
    > were *killed* in verification ? recorded in §5.5 so we don't repeat them. **License caveat:** the
    > code repos below are research releases; **independently confirm each repo's actual license is
    > MIT/Apache-2.0/BSD before vendoring any of it** (this survey did not verify every license file).
    > **Governance:** none of these techniques require training on paid-LLM outputs ? all are compatible
    > with our no-Gemini-training-data guardrail (§8).

    ### 5.1 Windowing & boundaries (TAL/TAS) ? the explicit-boundary machinery we lack
    The recurring lesson: modern action-localization models have an **explicit boundary mechanism**;
    our heuristic has none (it merges on a velocity+gap rule). The transferable *idea* is "predict a
    boundary and cut there," but the deep models are built for **hundreds+ of training videos**, so at
    n=6?16 they are **research bets**, not drop-ins.

    - **ASRF ? Action Segment Refinement Framework** (Ishikawa et al., WACV 2021, [arXiv:2007.06866](https://arxiv.org/abs/2007.06866)).
      A shared feature extractor feeds **two branches**: frame-wise classification + a **Boundary
      Regression Branch**; predicted boundaries refine/split the segmentation (+13.7% edit, +16.1%
      segmental F1 over MS-TCN-era SOTA on 50Salads/GTEA/Breakfast). **Why it fits:** the boundary head
      is exactly the missing "cut here" mechanism. **? Two caveats:** (1) ASRF's headline goal is to
      reduce over-segmentation (merge spurious fragments) ? the opposite of our over-*merge*; only the
      split-at-a-detected-boundary sub-mechanism transfers. (2) The claim that ASRF's boundary branch
      is a **model-independent post-hoc splitter** you can bolt onto any dense signal was **REFUTED**
      (§5.5) ? you must train the two-branch model, not graft the branch. **Effort/payoff:** research bet.

- **TriDet** (Shi et al., CVPR 2023, [arXiv:2303.07347](https://arxiv.org/abs/2303.07347)). A **Trident-head** models each boundary as a *relative probability distribution over local bins* (boundary = expected value over neighboring instants) ? built for imprecise/ambiguous boundaries. 69.3% avg mAP on THUMOS14 (> ActionFormer 66.8% at ~75% latency), **convolution-only (SGP), no self-attention** (attractive on a single local GPU). Ablation shows the Trident-head's gain concentrates at strict tIoU=0.7 ? it buys *boundary precision*. **Effort/payoff:** research bet (trained on 100s of videos); the portable piece is the relative-boundary idea.

- **ActionFormer** (ECCV 2022, [arXiv:2202.07925](https://arxiv.org/abs/2202.07925)). The reference single-stage anchor-free template: multiscale pyramid + local self-attention; classify every moment + regress its boundaries (71.0% mAP@0.5 THUMOS14). This is the family TriDet/TemporalMaxer/ CausalTAD extend, and the natural target if we ever graduate from heuristic+LogReg. **Research bet.**

- **TemporalMaxer** (CVPRW 2023, [arXiv:2303.09055](https://arxiv.org/abs/2303.09055)) ? **the most important small-data signal in this dimension.** It replaces ActionFormer's self-attention block with a **parameter-free max-pool** and *beats* it (67.7 vs 66.8 avg mAP) at **~4× fewer params**. The ablation is the takeaway: a **learned 1-D conv block is the WORST** (59.4, overfits redundant features); parameter-free pooling wins. **Why it fits:** direct evidence to **keep our scorer parameter-light** (the numpy LogisticRegression over a handful of features is the *right* choice at our scale, not a placeholder to rush past). **Effort/payoff:** a discipline (free) ? adopt the principle, not the model. *(Caveat: the overfitting evidence is about feature redundancy on a mid-size set, a directional inference to our small-n setting.)*

- **CausalTAD** (2024, [arXiv:2407.17792](https://arxiv.org/abs/2407.17792)) ? *steering-away note.* Causal-attention masking (attend only to past/present) won EPIC-Kitchens/Ego4D 2024 tracks, but it adds **no** boundary-split module. Useful later for *online/streaming* rally detection; **NOT** the over-merge fix. Listed so we don't mistake causal attention for the remedy.

- **Metrics (the actionable cross-cut):** the field leads with **segmental Edit (Levenshtein) score and F1@k**, not frame accuracy (MoF) or plain tIoU ? see §5.4.

5.2 Shuttle-trajectory ? rally signal processing ? published badminton systems already do what we lack

This is the highest-confidence, lowest-effort dimension: in-domain papers operationalize the exact boundary heuristics we're missing, in spirit license-compatible (re-confirm before vendoring code).

- **See et al. ? Service & End-of-Rally Detection in Badminton** (2024 Springer chapter [10.1007/978-3-031-69073-0_15](https://link.springer.com/chapter/10.1007/978-3-031-69073-0_15); 2025 SN Computer Science follow-on

[10.1007/s42979-025-03935-0](https://link.springer.com/article/10.1007/s42979-025-03935-0)).
298   **(a) Rally END = a sustained-inactivity rule:** the rally ends when **?80% of
consecutive frames
299   show no shuttle motion** (~**95% accuracy** over 187 rallies). **(b) Rally START
(service) = a
300   learned classifier** over court geometry + player poses (~26 features/frame × 12
frames ?
301   CNN/BiLSTM, **88.6%** accuracy). **Why it fits:** the inactivity rule is *precisely*
the
302   end-of-rally cut our windowing lacks ? a few lines, directly breaks 65 s mega-windows.
? Caveats:
303   the rule is a *termination*, not a *within-rally split*; results rest on paywalled
abstracts +
304   small test sets (35?187 rallies); the serve classifier needs the (default-OFF) person
cue + court
305   polygons. **Effort/payoff:** the inactivity rule = **low-effort high-confidence win**;
serve
306   classifier = medium.
307
308   - **Hsu, Yu & Cheng ? trajectory + swing fusion** (Sensors 2024, 24(13):4372,
309   [10.3390/s24134372](https://www.mdpi.com/1424-8220/24/13/4372)). Fusing TrackNet
trajectory with a
310   YOLOv7 swing detector via a Shot Refinement Algorithm raises hit detection **F1 72.3%
? 90.5%**
311   (over 69 BWF matches / 1582 rallies). Hits are found by **trajectory direction-
reversal / peak
312   detection** (convert (x,y)?(frame,y) wave peaks) then reconciled with the swing cue.
Why it fits:
313   validates our **late/score-level fusion** approach in-domain, and direction-
reversal/peak is a cheap
314   **per-window feature** marking contact events (a "rally has hit cadence" signal ? the
discriminator
315   a "shuttle-moving" heuristic lacks). **? Important caveat (the one 2-1 vote):** this
paper does
316   *within-rally stroke/hit* segmentation, **not** rally start/end ? "it splits merged
rallies" is *our
317   extrapolation*; frame it as "hit cadence plausibly helps locate rally boundaries," not
a proven
318   rally-splitter. **Effort/payoff:** the peak/direction-reversal feature = low-effort
win (feed the
319   scorer); full swing fusion = medium.
320
321   - **What best separates "rally" from "shuttle merely moving"** (synthesis): sustained-
inactivity
322   termination (See), hit cadence / direction-reversals (Hsu), net-crossings ?2 (our
Gate-A,
323   [`rally_gate.py`](../backend/pipeline/detectors/rally_gate.py)), two-sided court
occupancy (our
324   person features). None alone is sufficient; the spine (§4) says **feed them all to the
learned
325   scorer**.
326
327   ### 5.3 Small-data strategies ? most quality per label
328   - **Timestamp / sparse supervision** (Li et al., CVPR 2021,
[arXiv:2103.06669](https://arxiv.org/abs/2103.06669);
329   TAS survey Ding et al., [arXiv:2210.10352](https://arxiv.org/pdf/2210.10352)). Label
**one frame per
330   action** ? iteratively refine pseudo frame-labels ? *comparable* to full supervision
(50Salads:
331   timestamp 75.6% acc / 60.1 F1@50 vs full 83.7% / 70.1). **Why it fits:** sparse per-
rally timestamps
332   cost far less than dense boundary labels ? relevant if we ever train a frame-wise
model.
333   **? Caveats:** "comparable" hides a real **~8?10 pt residual gap**; annotation savings
are modest

(~35%, annotators still watch the whole clip); benchmarks are cooking/egocentric ? transfer to 1-D
shuttle trajectories is **unproven**. **Effort/payoff:** research bet to pilot.
- **Parameter-light scorer** ? see TemporalMaxer (§5.1): keep the numpy LogisticRegression; resist a
learned temporal net until the corpus is much larger. **Low-effort win (a discipline).**
- **? OPEN (the research could NOT verify these ? honest gap):** *active learning* (which video/segment
to label next via uncertainty/diversity/core-set) and the *teacher-student / distillation governance
boundary* for using Gemini as a weak/pseudo-labeler returned **no surviving verified claim**. Our
pragmatic, guardrail-safe stance (not from the literature ? from our own design): pick the next
video to label by **maximal local-vs-Gemini disagreement** (the §4.4 shadow signal) + **scorer
uncertainty** + **venue/fps diversity**; and treat **Gemini strictly as an inference-time
weak-reference / eval oracle ? never a training-data source for owned weights** (§8). These two
topics deserve their own grounded research pass before we lean on them.

5.4 Evaluation rigor at small n ? lead with segmental metrics
- **Frame accuracy (MoF) and plain tIoU are BLIND to over-segmentation** (TAS survey, Ding et al.,
[arXiv:2210.10352](https://arxiv.org/pdf/2210.10352); segmental-F1 origin Lea et al., CVPR 2017).
"The score can be high even when segments are fragmented." The over-merge that bit us is exactly
what tIoU-F1 hides (a giant window still *overlaps* real rallies). **Action:** lead the leaderboard
with the **segmental Edit (Levenshtein) score** (penalizes spurious ordering/fragmentation) **and
F1@{0.1,0.25,0.5}**, paired with our existing `segment_count_ratio` + the merge/split breakdown.
We already have F1@k, mAP@tIoU, segment-count-ratio, bootstrap CIs, paired sign-test, Platt/
isotonic + ECE/Brier ? the only *missing* primary metric is the **Edit score** (low-effort add).
- **? OPEN (not covered by a verified claim):** IAA / label-noise level in the golden set, and which
over-segmentation-robust loss (ASRF boundary/smoothing, MS-TCN truncated-MSE) best tolerates it at
small n. Measure IAA as the corpus grows (Roora has an A/A overlap plan); revisit robust losses
only when a frame-wise model is on the table.

5.5 Refuted claims (recorded so we do NOT repeat them)
1. **"ASRF's boundary branch is model-independent ? a post-hoc splitter applicable to any dense
segmentation output."** **REFUTED (0-3).** Implication: we cannot bolt ASRF's boundary head onto
our trajectory signal as a drop-in; the two-branch model must be trained together. ? port the
concept, don't expect a plug-in.
2. **"A GFL-inspired Boundary Distribution Regression head is the mechanism most relevant to splitting
merged windows."** **REFUTED (0-3)** (and the cited source had a suspect identifier). Do not adopt
this framing as established.

5.6 External due-diligence review (2026-06-17) ? owner-commissioned; verify before submission
An owner-commissioned external review (31 cited sources) of the

[`RALLY_DETECTION_QUALITY_REPORT.md`](RALLY_DETECTION_QUALITY_REPORT.md) checkpoint **validated the methodology** (the decoupled 2-layer design, the R2 metric, LOVO + paired-sign-test discipline, the consented-data + over-seg-aware-harness moat) and surfaced these **borrowable inputs**. These are *research inputs for later pickup* ? they add **no measured numbers**, and the citations are the review's own and **must be re-verified before any submission**.

- **Positioning ? the "first-mile" gap (highest-value).** Badminton stroke-classification SOTA (**BST** ? Badminton Stroke-type Transformer, CVPR-W 2026; **DSTA** ? Decoupling Spatio-Temporal Adapter) and the big datasets (**ShuttleSet** ~36,492 strokes; **Fine-Badminton** ~27,597 actions) all **silently assume a pre-trimmed rally clip already exists** ? they depend on a human having segmented the untrimmed match. Our system solves that ignored *prerequisite*. Reframe the paper as **"the missing untrimmed-video preprocessing pipeline that lets BST/MonoTrack run on raw footage"** ? a real, unoccupied gap and a far stronger narrative than "applied/systems."
- **R2 ? Action-Spotting / Precise-Event-Spotting (AS/PES).** Our asymmetric tolerance-match metric sits at the **TAL × AS intersection** (interval task, tolerance-based match). Grounding it in the AS/PES literature (SoccerNet, TenniSet, T-DEED, the PES survey [arXiv:2505.03991]) frames R2 as a deliberate methodological contribution, not an ad-hoc metric ? strengthens §5.4 and the report's §9.
- **Comparative baselines reviewers will require.** Benchmark against **MonoTrack** and **BST** on **ShuttleSet / Fine-Badminton** ? we currently have only a generic heuristic + the Gemini wrapper; domain-specific baselines are a hard publication requirement.
- **Scorer upgrade ? prefer TemporalMaxer.** For the post-logreg scorer (corpus-gated; NEXT_STEPS M0.4), **TemporalMaxer** (parameter-free temporal max-pool; ~2.8× fewer GMACs; robust at small-N, [arXiv:2303.09055]) fits the cheap/local/fast mandate better than attention-heavy ActionFormer.
- **Court polygons ? pose ? the rally-END lever.** Court masking is not only the person-cue spectator fix (§4 / `MULTI_SIGNAL_FUSION_PLAN.md`): once pose is isolated to on-court players it can resolve **ambiguous, occluded end-of-rally states** ? feeding directly into the **#1 remaining gap (the rally-END boundary)**, not just precision. Connect court-polygons to the R4/END-trim work.
- **Name the eval?serve lesson "pipeline distribution skew."** Downstream-cue value depends on the upstream proposal distribution (the Gate-A episode) ? a citable framing and candidate paper "lesson."
- **Citable Layer-1 anchor (public benchmark, NOT our footage).** WASB SBDT F1 95.6 (step=1) / 94.0 (step=3) vs MonoTrack 92.1, TrackNetV2 89.4, BallSeg 71.7 (WASB, BMVC 2023). Justifies the frozen detector choice; **keep distinct from our own, still-unmeasured footage domain-gap** (§2.1, a real open limitation the review also flags).

Sequencing: these are *publication-prep + post-current-idea* levers. The prerequisites remain corpus growth to n?16 and **multi-court isolated as its own reported slice** (§6 Tier-3

/ §7) ? do
    those first; the items above land once current windowing/fusion ideas are exhausted.

    ## 6. Prioritized roadmap (cheap ? durable)

    Every item is **its own measured PR**: default-OFF, scored on the §4 metric set over the golden set
    with honest stats (CI + sign-test), **re-confirmed on the served windowing** (the eval?serve gate),
    logged in [`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md). Ordered by the research's
    effort/confidence tiering. Each carries a **falsifiable success criterion** ? if it isn't met, the
    item does not promote (and that null is itself recorded).

    ### Definition of "bridging the gap"
    Local rally segmentation, measured **honestly LOVO-by-match on the golden set**, approaches the
    Gemini reference (today **0.93 single-court / 0.66 multi-court** tIoU-F1) ? but tracked primarily
    through the **over-segmentation-sensitive metrics** (segmental Edit + F1@k + low merge/split rate),
    since those, not bare tIoU-F1, capture our actual failure. A secondary, free signal is rising
    agreement-with-Gemini on the unlabeled pool (§4.4, shadow only).

    ### Tier 0 ? Make the failure measurable FIRST (do before any fix)
    - **E1 · Over-segmentation-aware leaderboard.** Add the **segmental Edit (Levenshtein) score** to
      `segmentation_metrics.py`; surface Edit + F1@{0.1,0.25,0.5} + `segment_count_ratio` + a
      merge-rate / split-rate breakdown (promote the `scratch/compare_local_vs_gemini.py` matching
      into a reusable `backend/eval/` helper) in the default `experiment.compare()` report; add the
      one-command rally-segmentation eval entrypoint (§4.5). **Success:** the current heuristic's
      over-merge shows up as a *number* (low Edit, high merge-rate) on the 6 golden videos, and a
      baseline row lands in the leaderboard. *(Low effort, high confidence ? research-backed §5.4.)*

    ### Tier 1 ? Low-effort, high-confidence wins
    - **W1 · Bounded windowing (the direct over-merge fix).** Add a `max_window_duration` cap and a
      sustained-inactivity split to `trajectory_to_action_windows` ? port See et al.'s rule (split /
      end a window when ?~80% of consecutive frames over a trailing window show no shuttle motion).
      Config-gated, default preserves current behavior. **Success:** segmental Edit + F1@k improve vs
      `CONTROL` on the golden set (?F1 CI excludes 0 **and** sign-test agrees) with **no served-windowing
      regression**; merge-rate drops materially. *(Win ? §5.2 See et al., ~95% end-of-rally acc.)*
    - **W2 · Rally-state features into the learned scorer (no special-casing).** Wire the already-built,
      default-OFF cues as scorer feature columns: **net-crossings** (Gate-A), **two-sided motion**, and a
      new cheap trajectory feature ? direction-reversal / peak count (Hsu) as a hit-cadence proxy;
      audio hit-rate when the hit detector lands. **Re-measure with W1 in place and on the served
      windowing** ? recall person was ?0.021 and audio +0.079 but *both CIs spanned 0* at

n=6 on the
pre-fix over-merged candidates; the picture may change once windows are bounded. **Success:** the
ablation (`ablation.py`) shows a cue with ?F1 CI clearing 0 + sign-test, surviving served re-check
? promote that cue; others stay default-OFF + retained (no deletion on a small-n null). *(Win/medium.)*
- **W3 · Keep the scorer parameter-light (a discipline, documented not coded).** Per TemporalMaxer's
ablation (learned temporal conv overfits; parameter-free wins), resist replacing the numpy
LogisticRegression with a learned temporal net until the corpus is much larger. Record this as a
standing decision. *(Win ? §5.1 TemporalMaxer.)*

Tier 2 ? Medium effort
- **M1 · Serve-start classifier ? rally-start boundary** (See et al. pose/court, 88.6%). Needs the
person cue + a **court-mask source** (manual polygon vs auto-homography ? *owner-gated* per
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) §9.1). **Success:** start-boundary
timing error drops on golden; promotes only if it lifts segmental F1@k without served regression.
- **M2 · Per-venue threshold calibration** against golden labels (the heuristic's footage-fragility ?
it scored 0.157 on multi-court). Tune `velocity_threshold` + `merge_gap` so local segment counts
track golden counts per venue. **Success:** multi-court F1 rises toward the Gen-0 head's 0.561
without harming clean-court F1. *(Velocity is already fps-normalized ? calibrate per-venue, not
per-fps.)*

Tier 3 ? Research bets (durable; corpus-gated)
- **R1 · Owned segmenter with an explicit boundary branch** (the ASRF/TriDet *idea* ? a boundary head
that predicts cut points ? ported into our owned TAL head, NOT the full deep model, and **trained**,
not grafted (§5.5 refutation)). Per [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md):
TAL head first, VLM later. **Gate:** the central unverified assumption is whether boundary-head
gains hold at our scale on trajectory features ? so **pilot only when the corpus is materially
larger (~?16?30 matches)**, and only after W1/W2 establish the heuristic ceiling. **Success:**
beats the tuned heuristic + LogReg on held-out matches (segmental Edit + F1@k), honestly LOVO.
- **R2 · Timestamp/sparse supervision** to get more labeled rallies per annotator-hour (§5.3) ? pilot
if/when we train a frame-wise model; research bet (residual gap, unproven on trajectories).

Throughout ? corpus growth + active learning + label quality
- **Grow the golden set 6 ? ~16** via Roora ([`GOLDEN_SET_VENDORING.md`](GOLDEN_SET_VENDORING.md)) ?
the single highest-leverage lever ([`NEXT_STEPS.md`](NEXT_STEPS.md)).
- **Active learning:** select the next videos to label by max **local-vs-Gemini disagreement** (§4.4)
+ scorer uncertainty + venue/fps diversity (our pragmatic stance; §5.3 flags the formal methods as
an open research pass).
- **Measure IAA / label noise** as the corpus grows (Roora A/A overlap); revisit over-segmentation-

```
485       robust losses only when a frame-wise model is on the table (§5.4 open question).
486
487     ### Dead-ends / do-NOT (so we don't relitigate)
488     - Don't treat **causal attention** (CausalTAD) as the over-merge fix ? it has no
boundary-split (§5.1).
489     - Don't expect **ASRF's boundary branch as a post-hoc drop-in** on our dense signal ?
refuted (§5.5).
490     - Don't adopt a **learned temporal net** at n=6?16 ? TemporalMaxer evidence says it
overfits (§5.1).
491     - Don't chase **bare tIoU-F1 / frame accuracy** ? blind to the over-merge (§5.4).
492     - Don't let **Gemini outputs become training data** for owned weights ? inference/shadow
only (§8).
493
494     ---
495
496     ## 7. Project: plan, tracking & definition of done
497
498     This roadmap is run as **one tracked project ? "Bridge the local?Gemini rally-detection
gap."** It
499     **completes when all committed tiers land AND a single quality number is attributed to
the effort**,
500     measured honestly and logged. This §7 is the living burn-down (update the status column
as PRs merge;
501     log each result in [`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md), numbers in
502     `eval_baselines/experiment_leaderboard.json`).
503
504     ### 7.1 The number (what we attribute to this effort)
505     Measured **leave-one-video-out, grouped by match, on the golden set** (grown to ?~16):
506     - **Headline:** multi-court rally-segmentation **tIoU-F1**, `baseline ? final`, plus
**gap-closure %**
507       = `(final ? baseline) / (Gemini ? baseline)`. Today: heuristic **0.157** ? Gemini
**0.66**
508       (Gen-0 head already **0.561**). **Success bar (proposed, owner-adjustable): close ?
50% of the
509       heuristic?Gemini multi-court gap** ? i.e. local multi-court F1 ? ~0.41, ideally
beating Gen-0's 0.561.
510     - **Primary lens (over-segmentation-sensitive, the actual failure):** segmental
**F1@0.25** +
511       **Edit-score** + **merge-rate** ? these must improve even if tIoU-F1 moves little.
512     - **Reported with honest stats** (bootstrap CI + paired sign-test); a result counts only
if it also
513       holds on the **served** windowing (eval?serve gate). The **baseline is frozen at E1**
before any fix.
514     - **Tracked headline = the n=6 golden *aggregate* tIoU-F1@0.5** (LOVO; the robust unit,
not a single
515       video) via `python -m backend.eval.rally_seg_eval`. The 0.157/0.66 figures above are
per-venue
516       *references*; computing an exact "% gap-to-Gemini closed" needs Gemini measured on the
*same* golden
517       set + metric ? **TODO** (don't mix the per-venue refs with the aggregate).
518
519     **Progress (cumulative, golden n=6 aggregate tIoU-F1@0.5):**
520
521     | after | F1 | merge_rate | ? vs baseline |
522     |---|---|---|---|
523     | E1 baseline (current heuristic) | 0.300 | 0.523 | ? |
524     | + W1 bounded windowing (cap=12, **default-OFF**) | **0.439** | **0.038** | **+0.139**
(6/0 sign-test) |
525     | + W2 net-crossings gate (mc=1, **default-OFF**) | **0.457** | 0.038 | **+0.157
cumulative** (W2 step +0.019, 6/0) |
526     | + W1.5 hard-cap fallback (**default-OFF**) | 0.444 | 0.000 | **+0.006 (n.s., CI spans
0)** ? neutral on golden; real-footage fix (mahadevpura-1 174s blob ? 24×?9s windows) |
527     | + W2 multi-feature scorer (person/audio) | _next (GPU)_ | | |
528
529     ### 7.2 Work items (status checklist)
```

```
530     Status: ? not started · ? in progress · ? done. (Each row = its own default-OFF,
measured PR.)
531
532     | ID | Deliverable | Tier | Depends on | Acceptance / promote criterion | Status |
533     |---|---|---|---|---|---|
534     | **E1** | merge/split-rate metric + F1@k surfaced + one-command `rally_seg_eval`
entrypoint; **froze baseline** (#185) | 0 | ? | over-merge shows up as a number on the 6 golden
videos | ? baseline F1 0.300, merge_rate 0.523 |
535     | **W1** | Bounded windowing ? `max_window_duration` cap + inactivity split (See et al.)
(#187) | 1 | E1 | seg F1@k ? vs baseline (CI?0 **and** sign-test), no served regression; merge-
rate ? | ? **?F1 +0.139 (6/0), merge_rate 0.523?0.038** (default-OFF) |
536     | **W1.5** | Hard-cap fallback (`window_hard_cap`) ? chop continuously-active windows at
the cap when no inactivity gap exists | 1 | W1 | golden ?F1 ?0 (or neutral) **+** fixes a real-
footage over-merge it can't otherwise cap | ? **golden ?F1 +0.006 (n.s.)**; real `mahadevpura-1`
174s blob ? 24×?9s; retained **default-OFF** (real-footage + true-cap rationale, not a golden
lift) |
537     | **W2** | Rally-state features (net-crossings ? #188 + **never-empty safeguard** ?;
two-sided motion, **direction-reversal/peak**, audio = follow-on) | 1 | E1, W1 | ?1 cue ?F1 CI
clears 0 + sign-test, survives served re-check ? promote; others retained default-OFF | ? **net-
crossings ?F1 +0.019 (6/0)**, gate now **do-no-harm** (never zeroes a video); multi-feature
scorer next (GPU) |
538     | **W3** | Keep scorer parameter-light (standing decision; TemporalMaxer evidence) | 1 |
? | documented; no learned temporal net until corpus ? now | ? |
539     | **M1** | Serve-start classifier ? rally-start boundary (pose/court) | 2 | W1, **court-
mask decision (owner-gated)** | start-boundary timing error ?; promote on seg-F1 lift, no served
regression | ? |
540     | **M2** | Per-venue threshold calibration vs golden | 2 | E1 | multi-court F1 ? toward
Gen-0 0.561 without harming clean-court | ? |
541     | **R1** | Owned segmenter with an explicit **boundary head** (port ASRF/TriDet *idea*,
trained ? not grafted) | 3 (stretch) | corpus ?~16?30, W1/W2 | beats tuned heuristic+LogReg on
held-out matches (seg Edit + F1@k), LOVO | ? |
542     | **R2** | Timestamp/sparse-supervision pilot (label-efficiency) | 3 (stretch) | R1 path
| measured label-efficiency win | ? |
543     | **C** | Golden corpus 6 ? ~16 (Roora) + active-learning selection + measure IAA |
ongoing | ? | corpus ?16; IAA recorded | ? (~6 now; ~10 by the weekend) |
544
545     ### 7.3 Sequencing
546     `E1` (metrics + baseline) ? `W1`/`W2`/`W3` (the high-confidence wins) ? `M1`/`M2` ?
`R1`/`R2`
547     (corpus-gated). `C` (corpus growth) runs in parallel throughout ? it is the single
highest-leverage
548     input and gates the Tier-3 bets.
549
550     ### 7.4 Definition of done
551     The project is **complete** when:
552     1. **Tier 0 + Tier 1 + Tier 2** work items are merged ? each a measured PR (default-OFF,
promoted only
553        on measured lift, re-confirmed on the served windowing).
554     2. **The headline number (§7.1) is recorded** in `QUALITY_ITERATIONS.md` + the
leaderboard, with honest
555        stats, on the grown golden set ? **and the gap-closure target is met, or the honest
null is recorded
556        with a reasoned go/no-go on Tier 3.**
557     3. Each landed item left the harness greener (no regression) and `QUALITY_ITERATIONS.md`
tells the story.
558
559     > **Tier-3 scope decision (owner-adjustable):** R1/R2 (the owned boundary-head model +
sparse
560     > supervision) are **corpus-gated stretch goals** ? the verified research says deep TAL
detectors risk
561     > overfitting at n=6?16. **Default:** the project *completes* at Tier 0?2 + an
attributed number, and
562     > Tier 3 spins out into the owned-model milestone
([`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md)).
563     > Fold Tier 3 into the DoD instead if you want the project to span the owned model.
```

```
564
565     ---
566
567     ## 8. Constraints & guardrails (non-negotiable) + cross-links
568
569     - **Licensing:** every dependency ? code, data, AND weights ? must be **MIT / Apache-2.0
/ BSD**.
570       Reject viral/NC/custom (Gemma-3, Llama-4 MAU cap, Commons-Clause).
571     - **Never train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs**
(terms/copyright);
572       paid LLMs are inference/serving/shadow-reference only. Also forbidden: public datasets
known to
573       be LLM-generated. **Exclude minors** from any training pool; per-user data needs
explicit opt-in.
574     - **Owned model:** TAL head FIRST (de-risked, trains in minutes, doubles as a pseudo-
label teacher +
575       honest baseline the VLM must beat), then Qwen3-VL (Apache-2.0) / InternVL3 (MIT) ? see
576       [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md).
577     - **Cross-links:** [`HOW_RALLY_DETECTION_WORKS.md`](HOW_RALLY_DETECTION_WORKS.md) ·
578       [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) ·
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) ·
579       [`ANALYZERS_AND_RECONCILERS.md`](ANALYZERS_AND_RECONCILERS.md) ·
[`ABLATION_LAYER.md`](ABLATION_LAYER.md) ·
580       [`EVALUATION.md`](EVALUATION.md) · [`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md) ·
581       [`GOLDEN_SET_VENDORING.md`](GOLDEN_SET_VENDORING.md) ·
[`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md).
582
```

25. assistant (2026-06-19T15:47:42.448Z)

This is a long doc. The key staleness signal in the task is "served operating point described as pre-flip" and "wrong default values". RALLY_QUALITY_RESEARCH.md §2.2 (lines 90-91) is now stale: it says **"because there is no `max_window_duration` cap and no inactivity/boundary split, a single window grows..."** ? but as of the R1 milestone (PR #204), `max_window_duration` defaults to 6.0 and `window_inactivity_end` to 1.5 (both ON in served config). This is the served operating point described as pre-flip.

However, I need to be careful: §2.2 is a **diagnosis of the original problem** that motivated the fix. The phrasing "because there is no cap and no split" describes the *original* state. But it's written in present tense ("has no `max_window_duration` cap") as a description of current code. Let me re-read lines 84-92 carefully. Line 84-85: "Active frames within `dead_time_merge_gap` (2.0 s) are merged" ? that's still true. Line 90-91 present-tense "there is no `max_window_duration` cap and no inactivity/boundary split" ? this is now FALSE in the served config (W1/R4 are default-ON).

This is a genuine contradiction. But §2 is explicitly framed as the historical diagnosis. The §7.2 tracker still shows W1 as "default-OFF" (line 535) and W1.5/W2 as "? in progress" ? these are now stale too given the R1 milestone flipped W1+R4 default-ON. Let me verify these are genuinely contradicted.

The §7.1 progress table (lines 519-528) and §7.2 status checklist describe W1 as default-OFF ? but PR #204 flipped W1 cap=6 + R4 to default-ON. Let me confirm the QUALITY_ITERATIONS doc and current-state to make sure the R1 milestone landed. The R1_MILESTONE_LAUNCH_REPORT.md (Status: LAUNCHED 2026-06-18, PR #204) plus models.py defaults confirm it. So RALLY_QUALITY_RESEARCH.md §7.2's "default-OFF" labels for W1 and the §7.1 cap=12 progress are stale relative to the shipped cap=6 default-ON milestone.

However, I must be cautious about over-reporting in a tracker that the task said not to relitigate. The task explicitly lists "served operating point described as pre-flip" and "wrong default values" as in-scope. Let me check the most concrete, defensible items. Let me verify the §2.2 default value claims precisely and whether RALLY_QUALITY_RESEARCH is in my slice (it is).

Let me confirm there's no `--tau-sweep`-related nuance and check the exact text in §2.2 / §7.1.

26. user (2026-06-19T15:47:44.078Z)

90:because there is **no `max_window_duration` cap and no inactivity/boundary split**, a single
150:**Signals available to L2 (built, default-OFF):** net-crossing count (`rally_gate.py`,
Gate-A),
298: **(a) Rally END = a sustained-inactivity rule:** the rally ends when **?80% of consecutive
frames
301: CNN/BiLSTM, **88.6%** accuracy). **Why it fits:** the inactivity rule is *precisely* the
304: small test sets (35?187 rallies); the serve classifier needs the (default-OFF) person cue
+ court
305: polygons. **Effort/payoff:** the inactivity rule = **low-effort high-confidence win**;
serve
321:- **What best separates "rally" from "shuttle merely moving"** (synthesis): sustained-
inactivity
414:Every item is **its own measured PR**: default-OFF, scored on the §4 metric set over the
golden set
437:- **W1 · Bounded windowing (the direct over-merge fix).** Add a `max_window_duration` cap
and a
438: **sustained-inactivity split** to `trajectory_to_action_windows` ? port See et al.'s rule
(split /
444: default-OFF cues as scorer feature columns: **net-crossings** (Gate-A), **two-sided
motion**, and a
450: ? promote that cue; others stay default-OFF + retained (no deletion on a small-n null).
(Win/medium.)
524:| + W1 bounded windowing (cap=12, **default-OFF**) | **0.439** | **0.038** | **+0.139** (6/0
sign-test) |
525:| + W2 net-crossings gate (mc=1, **default-OFF**) | **0.457** | 0.038 | **+0.157
cumulative** (W2 step +0.019, 6/0) |
526:| + W1.5 hard-cap fallback (**default-OFF**) | 0.444 | 0.000 | **+0.006 (n.s., CI spans 0)**
? neutral on golden; real-footage fix (mahadevpura-1 174s blob ? 24×?9s windows) |
530:Status: ? not started · ? in progress · ? done. (Each row = its own default-OFF, measured
PR.)
535:| **W1** | Bounded windowing ? `max_window_duration` cap + inactivity split (See et al.)
(#187) | 1 | E1 | seg F1@k ? vs baseline (CI?0 **and** sign-test), no served regression; merge-
rate ? | ? **?F1 +0.139 (6/0), merge_rate 0.523?0.038** (default-OFF) |
536:| **W1.5** | Hard-cap fallback (`window_hard_cap`) ? chop continuously-active windows at the
cap when no inactivity gap exists | 1 | W1 | golden ?F1 ?0 (or neutral) **+** fixes a real-
footage over-merge it can't otherwise cap | ? **golden ?F1 +0.006 (n.s.)**; real `mahadevpura-1`
174s blob ? 24×?9s; retained **default-OFF** (real-footage + true-cap rationale, not a golden
lift) |
537:| **W2** | Rally-state features (net-crossings ? #188 + **never-empty safeguard** ?; two-
sided motion, **direction-reversal/peak**, audio = follow-on) | 1 | E1, W1 | ?1 cue ?F1 CI
clears 0 + sign-test, survives served re-check ? promote; others retained default-OFF | ? **net-
crossings ?F1 +0.019 (6/0)**, gate now **do-no-harm** (never zeroes a video); multi-feature
scorer next (GPU) |
552:1. **Tier 0 + Tier 1 + Tier 2** work items are merged ? each a measured PR (default-OFF,
promoted only

27. assistant (2026-06-19T15:48:03.460Z)

Now I have a clear picture of the staleness in RALLY_QUALITY_RESEARCH.md. Lines 90, 524-526, 535 describe W1 (and R4) as default-OFF / no cap, but the R1 milestone (PR #204) flipped W1 cap=6 + R4 default-ON.

However, I need to be careful. This is a "tracked project" doc with a §7.2 burn-down. The §7.1/§7.2 trackers show pre-milestone state (cap=12, default-OFF, W1 at n=6). The R1_MILESTONE_LAUNCH_REPORT.md is the authoritative record of the flip (cap=6, default-ON, n=13). So the RALLY_QUALITY_RESEARCH.md §2.2 present-tense claim and §7.1/§7.2 trackers contradict the shipped code in `models.py`.

But the §7 tracker rows are explicitly historical progress entries (E1 baseline, then W1 step). A progress table recording "+W1 (cap=12, default-OFF) ? 0.439" is a *historical measurement record*, not a claim about current defaults ? recording that the W1 PR landed default-OFF at the time is factually what happened. The owner's convention treats §7.2 as a living burn-down that should be updated. But I should be conservative and only flag the clearest code-contradictions, not relitigate the tracker's historical rows.

The clearest, unambiguous code-contradiction is **§2.2 line 90-91**: the present-tense diagnosis "there is no `max_window_duration` cap and no inactivity/boundary split" directly contradicts `models.py` where `max_window_duration = 6.0` and `window_inactivity_end = 1.5` are now the served defaults. This is a present-tense factual statement about the code that is now false.

Let me now focus on the EVALUATION.md and the R2 doc more precisely. Let me check the R2 doc §0/§2.3 numbering and the "n=9 corpus" claim vs the milestone n=13. The R2 doc §2.3.3 reference ? the section is numbered as item 3 within §2.3, but text refers to "§2.3.3" ? this is an internal anchor, not a code claim, skip.

Let me check one more concrete thing: the R2 design doc references over-seg guard at "§2.3.3" and the milestone numbers. These are cross-consistent. Let me verify the EVALUATION.md "greedy one-to-one" matching is still accurate (metrics.py line 50 confirms greedy). EVALUATION.md describes the *legacy* `run_eval.py`/`harness.py` path which is real and unchanged ? no staleness there.

Now let me check whether the EXPERIMENT_HARNESS §3.8 line-number citations are worth flagging. Line citations like `default_segmenter` (:65) vs actual :118 are drifted, but the task scope is "metric/windowing doc staleness vs code" and these are a code-map's inherent line drift. The substantive claim "registered (6): motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid, fusion_hybrid" ? I confirmed exactly 6 registrations. That's accurate. I'll skip the line-number drift as low-value.

Let me verify one final thing ? the R2 doc's §3 says "Hungarian in the impl" and tolerance_metrics.py confirms Hungarian-via-scipy. Consistent. And the R2 status line says corpus "n=9" in the refinement note ? that's a dated historical measurement (2026-06-17), not a current-default claim, so not stale.

Let me make my final determination on the strongest items. Let me re-read the exact §2.2 text to copy verbatim.

28. user (2026-06-19T15:48:04.742Z)

```
86
87      **The conceptual flaw: "the shuttle is moving" is treated as "a rally is in progress."**
During
88      dead time the shuttle is still visible and gently moving (being picked up, walked back,
knock-ups,
89      slow serve prep) ? those frames read as *active*. The 2 s merge then bridges the brief
gaps, and
90      because there is **no `max_window_duration` cap and no inactivity/boundary split**, a
single
91      window grows until the shuttle finally goes invisible ? which, at 96?99 % visibility, is
rarely.
92      Result: rally + dead-time + rally + ? collapse into a few mega-windows.
93
```

29. assistant (2026-06-19T15:48:18.772Z)

This §2.2 passage is the diagnosis of the *original* failure mode. The phrasing "because there is no `max_window_duration` cap and no inactivity/boundary split" is genuinely describing the pre-fix code. Since W1/R4 are now built AND default-ON, this present-tense statement contradicts the code. However, this is clearly a *historical diagnosis paragraph* explaining why the bug existed ? rewriting it to past tense is the minimal honest fix, but I want to be careful not to over-edit a narrative that's intentionally describing the original state.

Given the task's explicit focus ("served operating point described as pre-flip"), this qualifies. The cleanest surgical fix is to mark it as the original/pre-fix state. Let me make it a low-severity finding with a minimal edit that adds past-tense framing.

Now let me reconsider the EXPERIMENT_HARNESS.md §3.8 claim more carefully ? it has a concrete factual error worth checking: it cites `IndexingConfig` knobs with line numbers AND says "The `min_crossings` knob now lives on `FusionConfig`". Let me verify min_crossings is on FusionConfig (confirmed: models.py line 105, `min_crossings: int = Field(default=0, ge=0)` inside FusionConfig). That's accurate. The line numbers (`:65`, `:72` etc.) are stale but it's a

navigation map.

Let me also double-check the R2 design §5 "Implementation plan" `--tau-sweep` claim once more ? since the doc status is "IMPLEMENTED" and §5 is "Implementation plan (pure, offline, additive)", the bullet "add `--tau-start/--tau-end/--tau-sweep`" describes the wiring. `--tau-start`/`--tau-end` exist (lines 467-470 of rally_seg_eval.py), but `--tau-sweep` does NOT. Since the doc presents this as done implementation and the other two flags landed, the `--tau-sweep` is an inaccuracy. But it's in a plan section and the sweep capability exists via `tolerance_sweep()`. I'll flag it as low severity since the specific named CLI flag doesn't exist while its siblings do, which could mislead someone looking for it.

Let me verify there's truly no tau-sweep CLI flag and no `tolerance_sweep` call in rally_seg_eval (the doc implies the sweep is surfaced).

30. user (2026-06-19T15:48:19.880Z)

No matches found

31. assistant (2026-06-19T15:48:35.129Z)

Confirmed: `rally_seg_eval.py` neither exposes a `--tau-sweep` flag nor calls `tolerance_sweep`. So the R2 doc §5 claim that the wiring adds `--tau-sweep` and §6 DoD "reports the R2 decomposition + ?-sweep" is not implemented in the `rally_seg_eval` entrypoint.

Let me re-check §6 DoD wording: "rally_seg_eval reports the R2 decomposition + ?-sweep next to tIoU". The decomposition IS reported (P/R/F1@? + boundary error + over-seg). But the ?-sweep is NOT in rally_seg_eval's output. This is a real, concrete contradiction worth flagging.

Let me also confirm the R2 doc §0/§2.3 anchor "§2.3.3" ? text at line 81 says "Implemented as `tolerance_metrics.over_seg_guard`" which is accurate. Fine.

Now, one more: the R2 doc §5 also says `boundary_error_report(matches)` takes matches, but the actual signature is `boundary_error_report(preds, gts)` (tolerance_metrics.py:122). This is a signature contradiction. Let me confirm ? §5 line 143: "`boundary_error_report(matches) -> {?start, ?end: signed/abs median+IQR}`". The actual function takes `(preds, gts)` and computes over best-overlap matches internally (the doc's own §0 refinement note even explains it's NOT over the within-? matches). So `boundary_error_report(matches)` is a stale signature. But §5 is the "implementation plan" written before coding ? the refinement note at the top (lines 10-15) already corrects the *semantics*. Still, the signature `(matches)` is literally wrong vs `(preds, gts)`. Low severity, but a real contradiction. However it's in a forward-looking "implementation plan" sketch. I'll flag it as low.

Let me finalize. Let me also verify the R2 doc §5 `over_seg_report` signature claim: "`over_seg_report(preds, gts) -> {split_count, merge_count}`" ? matches tolerance_metrics.py:150. Good. And `match_1to1(preds, gts, ?_start, ?_end) -> (matches, P, R, F1)` ? matches line 88. Good.

Now I have my findings. Let me make the edits and report. The strongest, clearest items:

1. **RALLY_QUALITY_RESEARCH.md §2.2** ? present-tense "no `max_window_duration` cap and no inactivity/boundary split" contradicts shipped defaults (cap=6, inactivity_end=1.5 default-ON per PR #204).
2. **R2_EVAL_METRIC_DESIGN.md §5** ? claims `--tau-sweep` flag added to rally_seg_eval; no such flag exists (only `--tau-start`/`--tau-end`).
3. **R2_EVAL_METRIC_DESIGN.md §6** ? DoD says rally_seg_eval reports the ?-sweep; it does not.
4. **R2_EVAL_METRIC_DESIGN.md §5** ? `boundary_error_report(matches)` signature wrong (actual: `(preds, gts)`).

Let me apply the edits to the docs.

32. user (2026-06-19T15:48:39.768Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\RALLY_QUALITY_RESEARCH.md has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-19T15:48:42.246Z)

Now the R2 doc edits. First the `--tau-sweep` claim in §5.

34. user (2026-06-19T15:48:45.971Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\R2_EVAL_METRIC_DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

35. user (2026-06-19T15:48:51.291Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\R2_EVAL_METRIC_DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

36. user (2026-06-19T15:48:55.318Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\R2_EVAL_METRIC_DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

37. assistant (2026-06-19T15:49:01.382Z)

All edits applied. Now let me return the structured findings. Let me capture the exact verbatim original text for each (before my edits, as required by the schema ? `stale_text` must be the text to replace).

38. user (2026-06-19T15:49:19.110Z)

Structured output provided successfully

39. assistant (2026-06-19T15:49:31.750Z)

I audited the eval/metric/windowing docs in my slice against the code and applied four verified surgical fixes. Summary of findings:

docs/RALLY_QUALITY_RESEARCH.md (medium) ? §2.2's present-tense diagnosis "there is no `max_window_duration` cap and no inactivity/boundary split" describes the served operating point as pre-flip. The R1 milestone (PR #204, R1_MILESTONE_LAUNCH_REPORT.md) flipped both default-ON: `backend/config/models.py:139` (`max_window_duration: float = 6.0`) and `:153` (`window_inactivity_end: float = 1.5`), and `windowing.py::_served_preset_from_defaults()` lifts them into SERVED. Reworded to past tense + a note that W1/R4 are now default-ON.

docs/R2_EVAL_METRIC_DESIGN.md ? three contradictions vs `backend/eval/rally_seg_eval.py` and `tolerance_metrics.py`:
- §5 (low): claims `--tau-sweep` was wired in; only `--tau-start`/`--tau-end` exist as CLI flags (rally_seg_eval.py:467/469), no `--tau-sweep`.
- §6 DoD (low): claims `rally_seg_eval` reports the ?-sweep; it reports the decomposition but never calls `tolerance_sweep`.
- §5 (low): wrong signature `boundary_error_report(matches)`; actual is `boundary_error_report(preds, gts)` (tolerance_metrics.py:122).

I deliberately did **not** flag: EVALUATION.md (describes the unchanged legacy `run_eval.py`/`harness.py` greedy-1:1 path, matches `metrics.py`); EXPERIMENT_HARNESS.md §3.8 line-number drift in its code-map (navigation map, substantive "6 registered segmenters" + "`min_crossings` now on `FusionConfig`" claims are accurate); and the RALLY_QUALITY_RESEARCH.md §7 burn-down's historical progress rows (cap=12/default-OFF/n=6 are dated records of what landed at the time, an owner-gated tracker, not a present-tense code claim).

Files edited (absolute paths):
- C:\Users\avidu\Projects\badminton-highlight-indexer\docs\RALLY_QUALITY_RESEARCH.md
- C:\Users\avidu\Projects\badminton-highlight-indexer\docs\R2_EVAL_METRIC_DESIGN.md